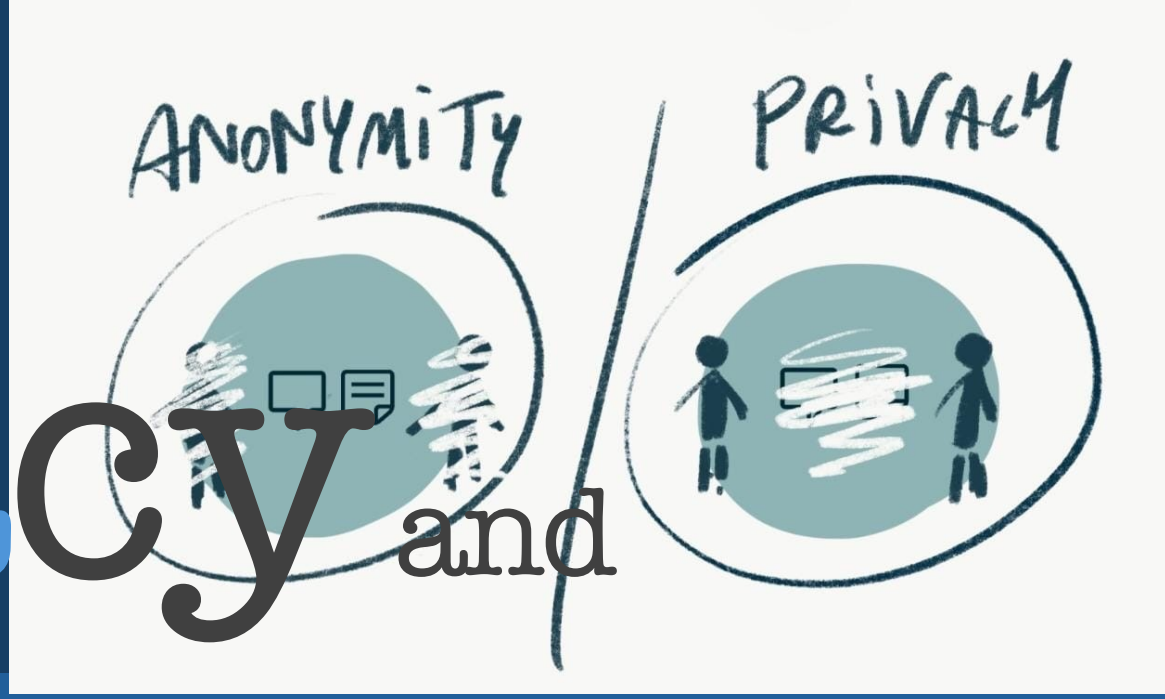
Privacy and Anonymity



Daniyal Pirwani

Online communication

Privacy: The ability to control the collection, use and distribution of personal information in online environments.

1- Involves **data protection**, **consent**, and **control**
2 - Does not necessarily mean hiding your identity
3 - Example: Encrypted messaging with your real name visible

Anonymity: The state of being unidentifiable within a set of subjects, such that your actions cannot be linked back to your identity.

1 - Focus on **unlinkability** and **non-identifiability**
2 - Typically requires technical systems (e.g., Tor)
3 - Example: Posting to a forum without revealing your IP or identity

Pseudonymity: The use of a consistent identifier (pseudonym) that is not directly linked to your real-world identity.

1 - Allows **reputation-building** without identity disclosure
2 - Can be **reversible** (linkable) or **persistent but unlinkable**
3 - Example: A Reddit user account with no real name attached

Privacy

Privacy: The ability to control the collection, use and distribution of personal information in online environments.

1- Involves **data protection**, **consent**, and **control**
2 - Does not necessarily mean hiding your identity
3 - Example: Encrypted messaging with your real name visible

Example:

Facebook, Instagram, Snap accounts ➔ Upload personal info – sign up

However.. use private information for cross-domain tracking??

* * **Privacy concern** * *

Privacy

Privacy: The ability to control the collection, use and distribution of personal information in online environments.

1- Involves **data protection**, **consent**, and **control**
2 - Does not necessarily mean hiding your identity
3 - Example: Encrypted messaging with your real name visible

HOME > TECHNOLOGY BUSINESS NEWS

New report finds Facebook users are tracked by thousands of companies

On average, each participant in the study had their data sent to Facebook by more than 2,000 companies.

Ian Krietzberg • Jan 17, 2024 4:01 PM EST

Home > Tech > Tech Industry

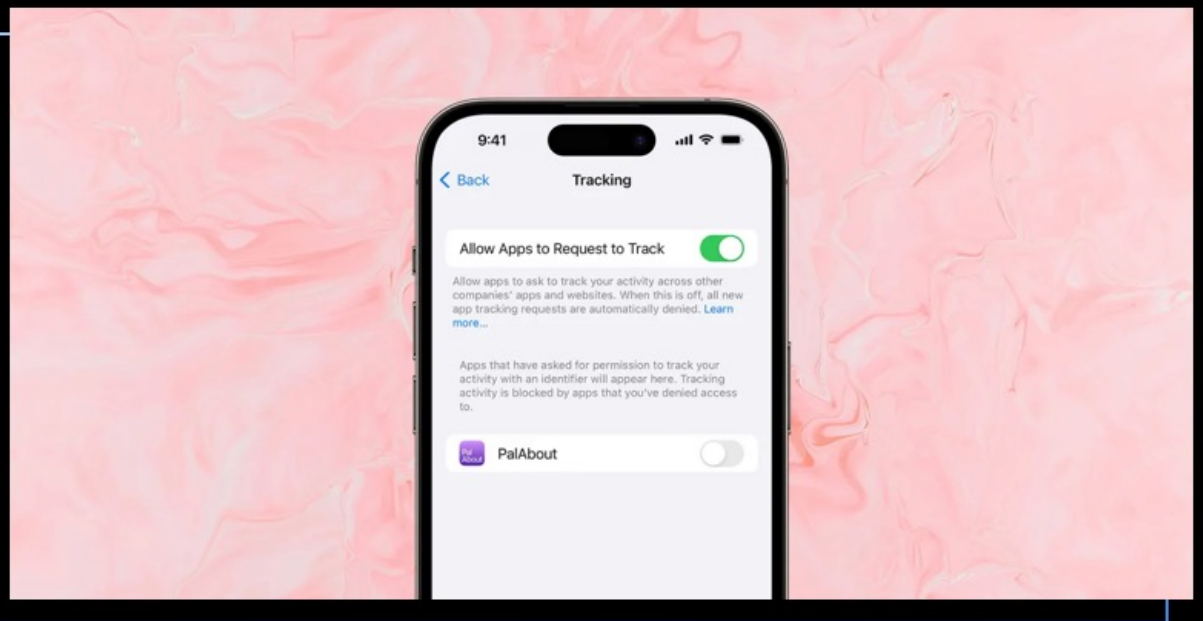
Google is tracking your digital fingerprints again

The tech company makes a U-turn in its privacy promises.

By Chase DiBenedetto on December 20, 2024

Apple bizarrely fined $162M for App Tracking Transparency after advertisers complained

Ben Lovejoy | Mar 31 2025 - 5:23 am PT 17 Comments



Privacy

Why care?

> “Arguing that you don't care about the right to privacy because you have nothing to hide is no different than saying you don’t care about free speech because you have nothing to say."
>
> **-Edward Snowden**

Privacy

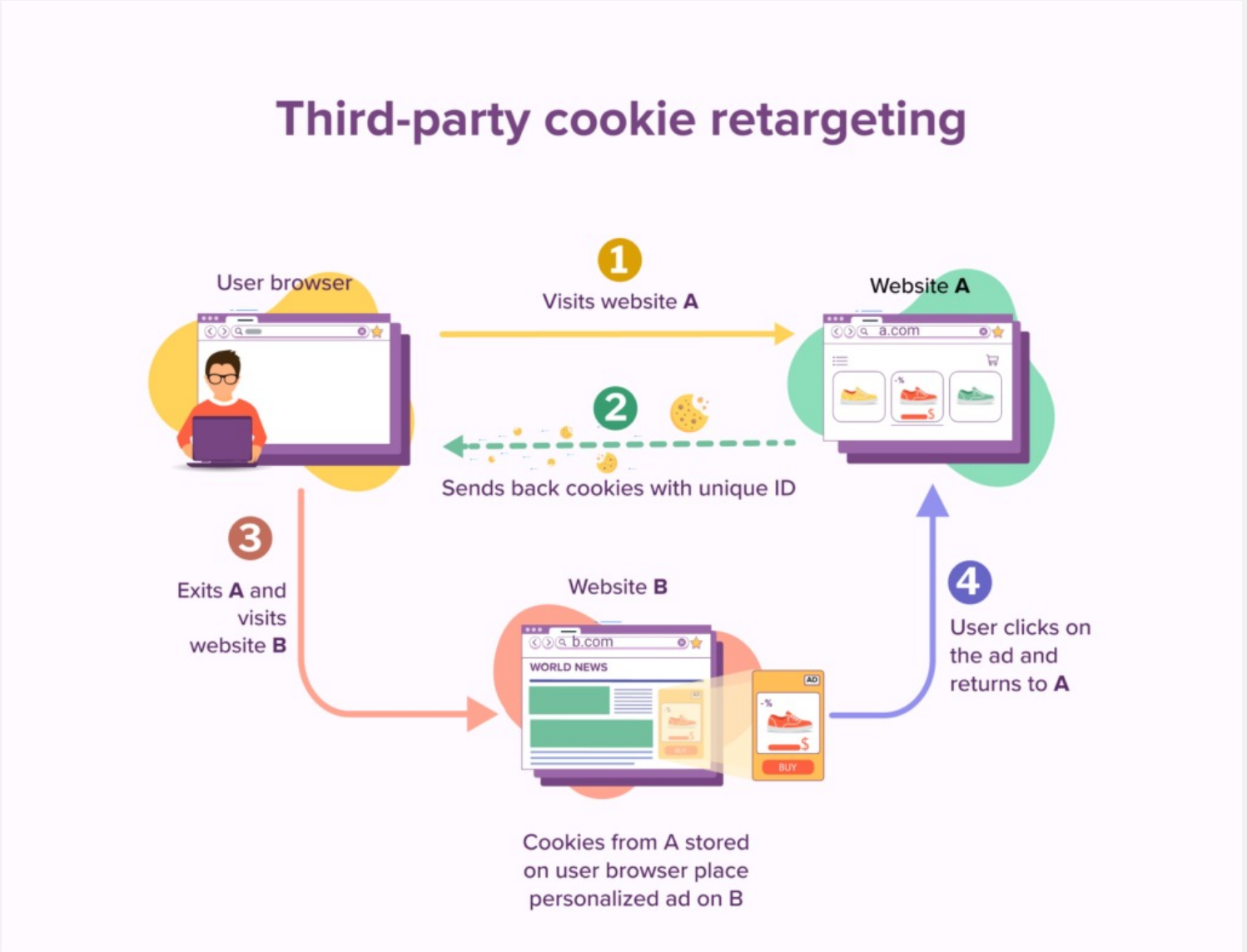
Why care?

> “Arguing that you don't care about the right to privacy because you have nothing to hide is no different than saying you don’t care about free speech because you have nothing to say."
>
> **-Edward Snowden**

“I’m very careful with my accounts online”

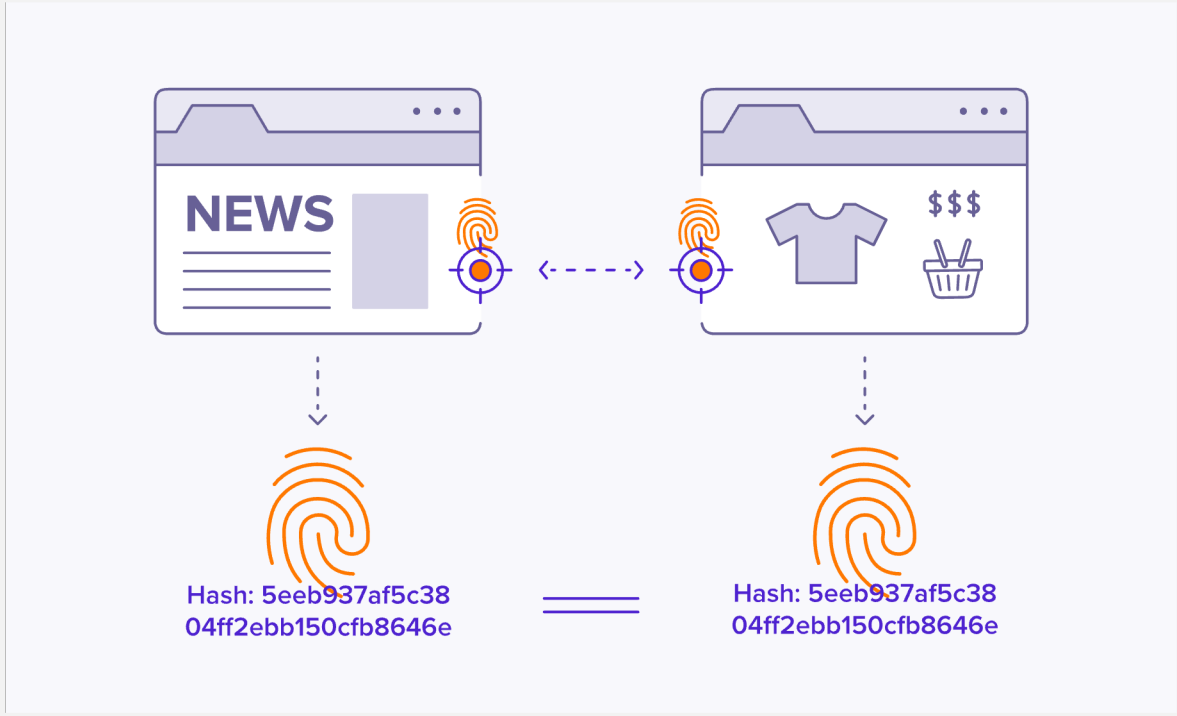
Privacy

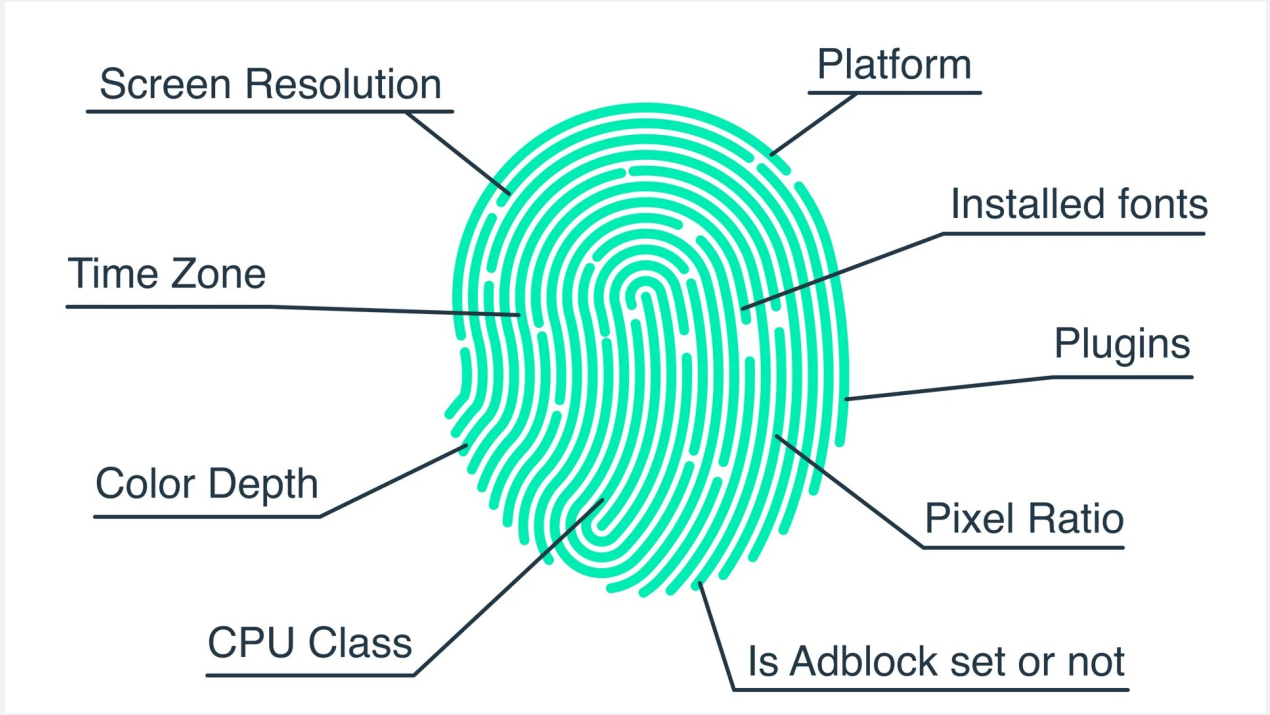
"I'm very careful with my accounts online"



Privacy

“Umm.. I use incognito”





Browser fingerprinting

1. **Canvas fingerprinting**
   a. HTML5 canvas elements
   b. Fonts, font size, background colors → identify your browser
2. **WebGL fingerprinting**
   1. On-screen rendering capabilities
   2. Devices with different resolutions process elements differently
   3. Invisible WebGL elements render at different rates, uniquely identifying users.
   4. Invisible tracking pixels.
3. **AudioContext fingerprinting**
   1. Audio APIs allows websites to play low-frequency sounds to measure how the audio is processed and test the ways the device plays the sound.
   2. Allows for another unique identification metric.
4. **Battery fingerprinting**
   1. Battery status information is unique (we all charge and use phones at different rates, leading to different battery cycles.
   2. Battery health differences lead to unique identification

[1] https://soax.com/blog/browser-fingerprinting-what-is-it-and-how-does-it-work
[2] https://www.zenrows.com/blog/browser-fingerprinting#webgl-fingerprinting/

Browser fingerprinting

1. **Canvas fingerprinting**
2. **WebGL fingerprinting**
3. **AudioContext fingerprinting**
4. **Battery fingerprinting**

“We all aren’t that unique”

In isolation, these features need not be enough to uniquely identify users. However, when combined together, they give trackers enough unique fingerprints to track users across the internet.

Solution?

Disable JavaScript?

Not realistic.. The internet relies on JavaScript and without it content becomes unusable.

VPNs!

Tracker don't need your IP address. Well.. Trackers don't even need your name or email address, of course it helps, but trackers and advertisers have adapted to growing user privacy concerns.

AdBlockers:

Can defend against many fingerprinting scripts, but not against the techniques we discussed earlier.

Modern Browsers (Firefox, Tor, Brave):

Systematically subvert scripts to resist fingerprint (spoof normalized data, makes fingerprinting hard)

Anonymity

Anonymity: The state of being unidentifiable within a set of subjects, such that your actions cannot be linked back to your identity.

1 - Focus on **unlinkability** and **non-identifiability**

2 - Typically requires technical systems (e.g., Tor)

3 - Example: Posting to a forum without revealing your IP or identity

Example:

Accessing websites/services with **no identification information.**

No IP address

No PII

No fingerprints

Internet access **must protect user identity**, should make it infeasible to **link any user to any website** (and sometimes vice versa)

Threat model

Who are we hiding from?

1. **Any** network entity (ISPs, nation-state censors, government monitors)
2. May include the visiting website (if you order something from Amazon, you need to give your PII, but the traffic must not contain identifiers that link you to Amazon.
3. However, you may need to trust some intermediary entity or system to anonymize the traffic on your behalf.
4. **Therefore: * * ALWAYS USE TLS * *.**
5. Why? ➔ Adversary may offer the services to you, therefore the entity or system you may be using **MAY BE ADVERSARY CONTROLLED.**

Are VPNs sufficient?

Short answer: No

Can you trust a VPN?

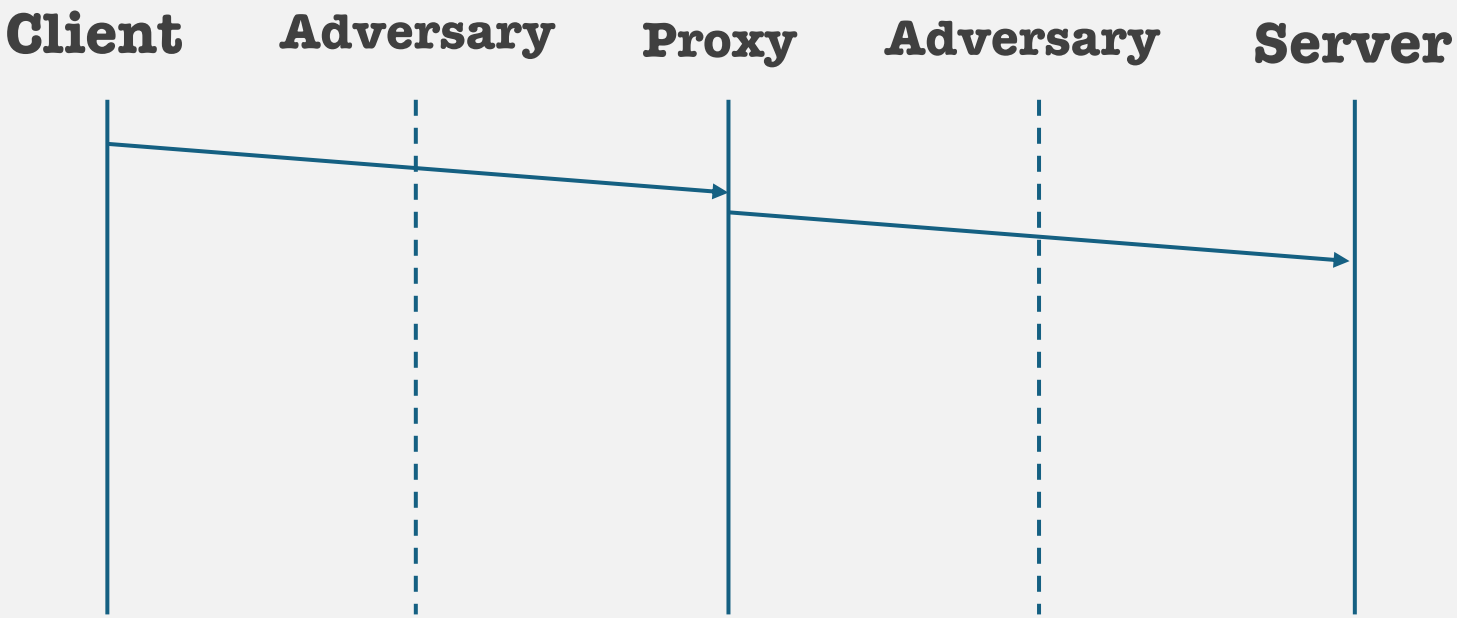
What if it is adversary controlled?

What if it gets subpoenaed if you are in a censored region?

“I use TLS”

TLS is insufficient, network traffic analysis. Timeseries analysis.

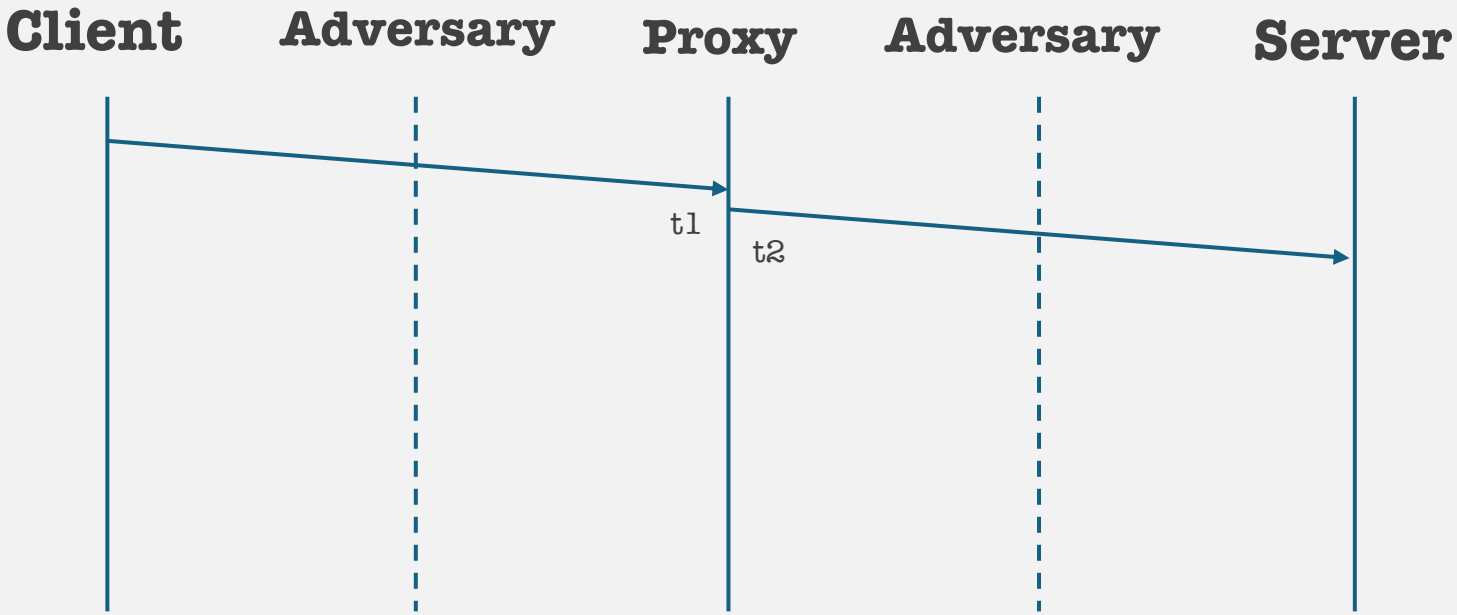
Traffic analysis



If the adversary controls the VPN/Proxy, ggwp

If the adversary is sniffing on both sides of the VPN, the adversary can deduce the src/dst through traffic analysis

Traffic analysis

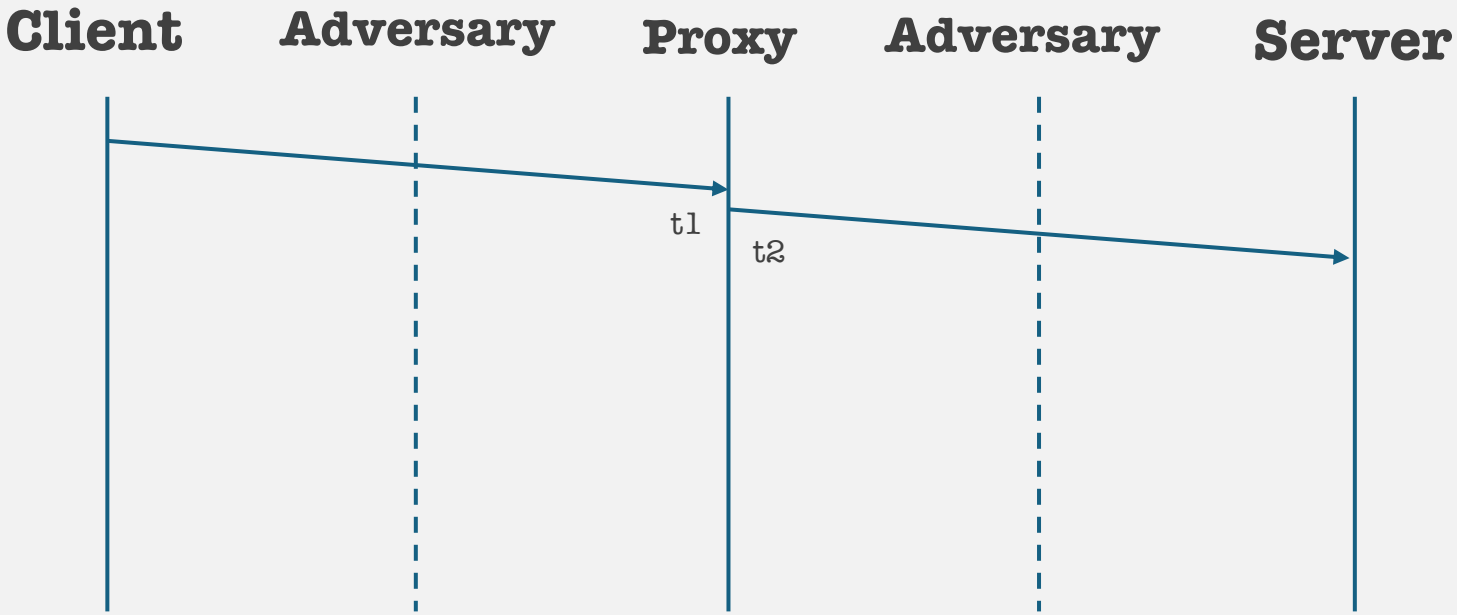


t1 : Time packet arrives at a proxy :: src = client, dst = proxy

t2 : Time packet leaves the proxy :: src = proxy, dst = server

t2 - t1 : relationship between pkt1 and pkt2

Traffic analysis



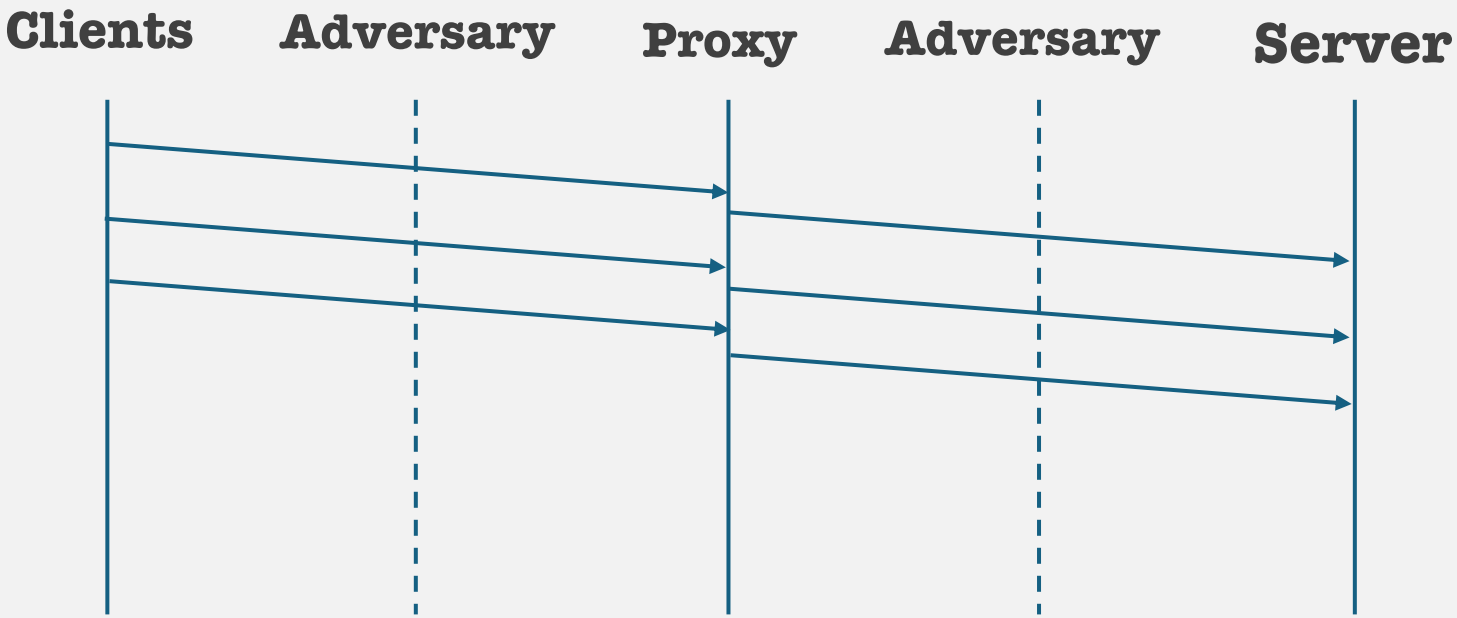
t1 : Time packet arrives at a proxy :: src = client, dst = proxy

t2 : Time packet leaves the proxy :: src = proxy, dst = server

t2 - t1 : relationship between pkt1 and pkt2

“What if multiple users are using the same proxy? That will anonymize the traffic.” No.

Traffic analysis



Proxies prioritize efficiency:

FIFO, once a packet arrives, it is immediately sent

As a result, packet arrival and departure times are sufficient to link clients to destinations.

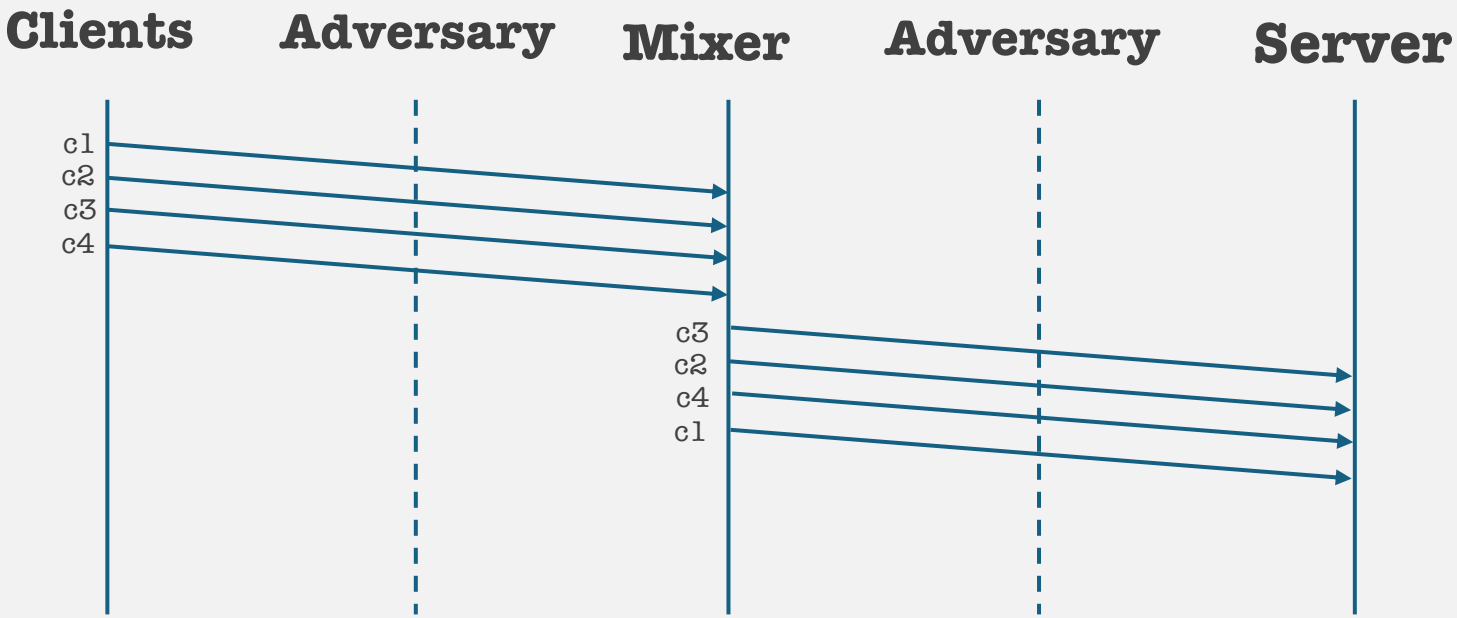
Mixer networks

Mixer networks are special purpose proxies that prioritize anonymity over efficiency:

Wait for queue-up, once a packet arrives, the mixer will queue it until sufficient packets arrive

When multiple packets from different senders queue up, the mixer randomly selects packets to send to the destinations.

Mixer networks



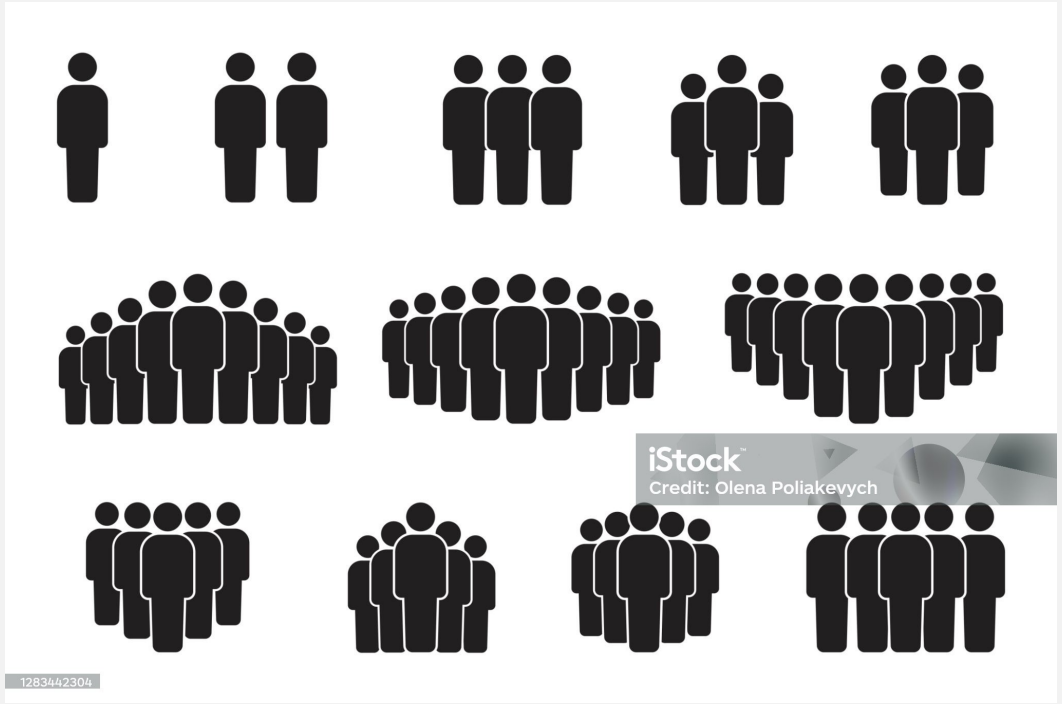
Mixer networks are special purpose proxies that prioritize anonymity over efficiency:

Wait for queue-up, once a packet arrives, the mixer will queue it until sufficient packets arrive

When multiple packets from different senders queue up, the mixer randomly selects packets to send to the destinations.

Makes traffic analysis based on time unreliable.

Introduces a key idea:



Mixers (like all other anonymous communication systems) rely on one key idea:

Introduces a key idea: Anonymity set

Mixers (like all other anonymous communication systems) rely on one key idea:

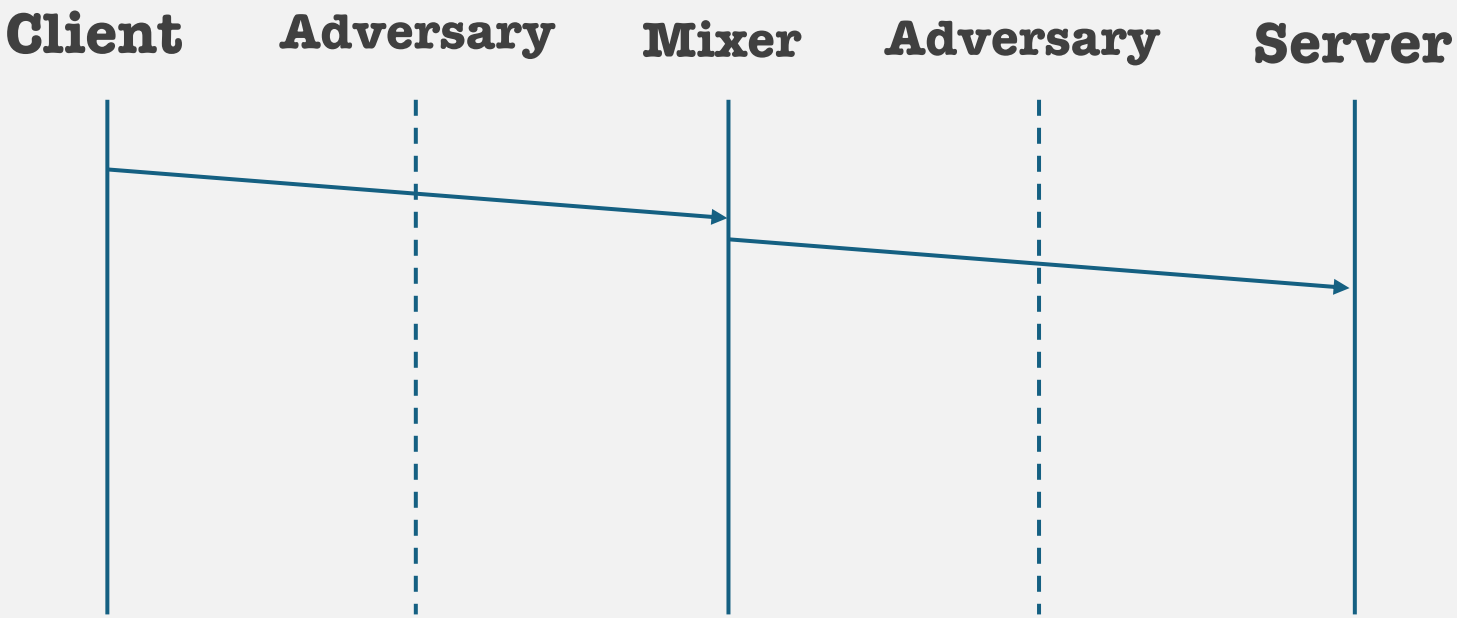
Anonymity Set:

Crowd to hide in.

In the context of anonymity, the set of subjects who could have sent something form the anonymity set.

Difficult to find who exactly sent what, since each request is hidden behind a crowd of requests.

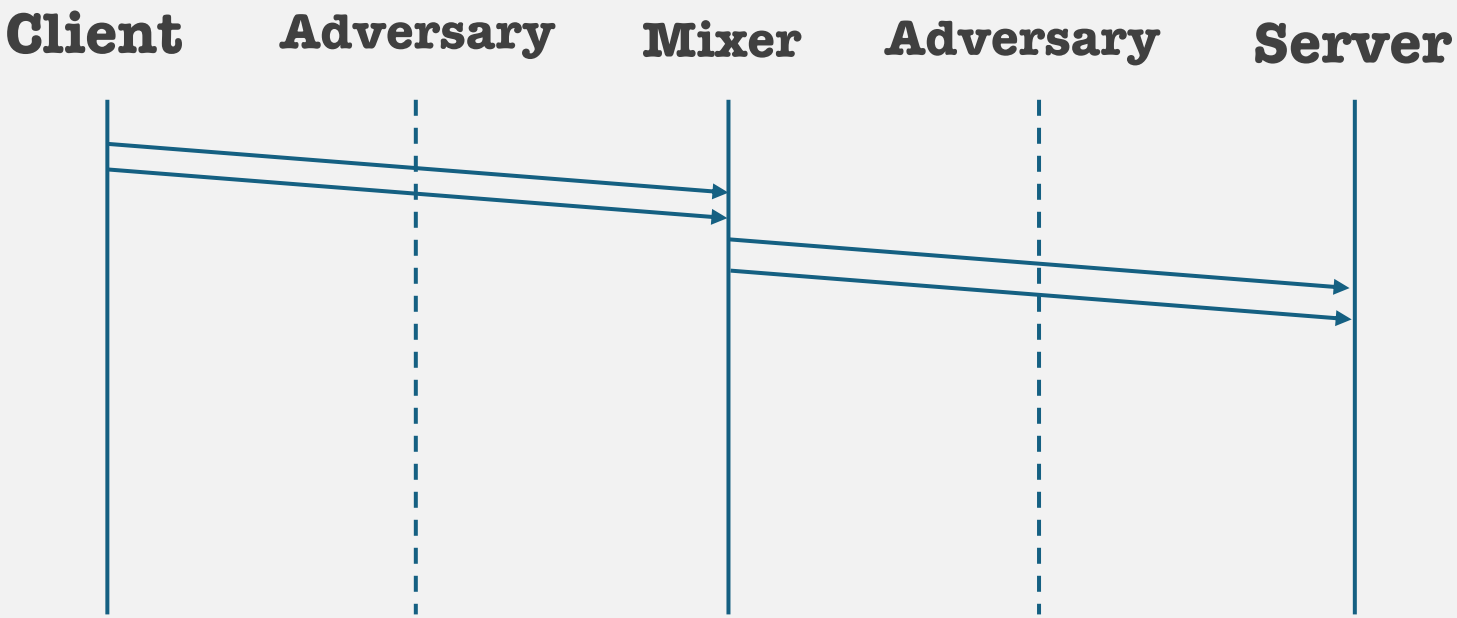
Anonymity set



Only one user..

Mixer is useless.. What is it mixing? Air?

Anonymity set

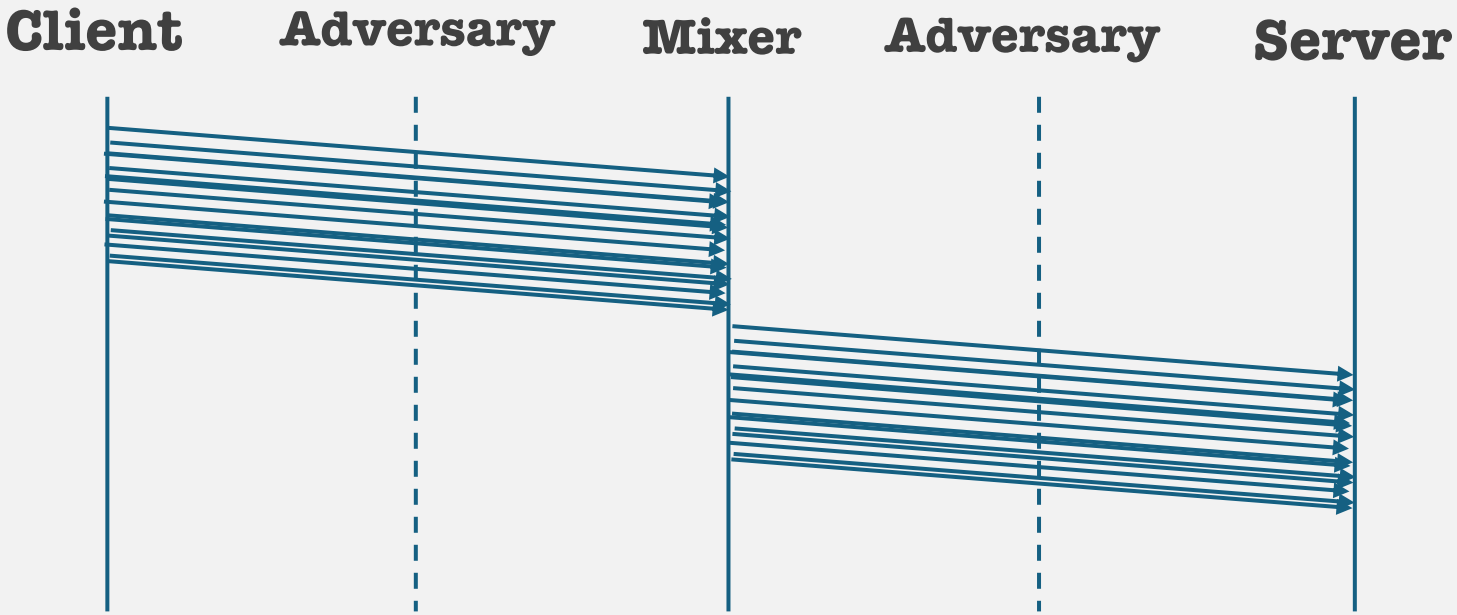


Two users?

Just grab em both, ask questions later.

Why? The cost of blocking is **low**. Selecting both using the same service is **cheap**.

Anonymity set



Million users?

.. Oh no.

Why? The cost of blocking is **high**. Selecting **all** using the same service is **expensive**.

We will revisit this concept in the next lecture. I will not tell you what we call this rn.

Introduces a key idea: Anonymity set

Definition: The **anonymity set** is the group of all possible subjects (usually users or entities) **who could have performed a given action**. From the perspective of an observer (like an attacker), each member of this set is indistinguishable from the others regarding that action.

In Simpler Terms: If someone sends a message anonymously, the anonymity set includes all the people who *could have* sent that message. The bigger this group, the harder it is to figure out who the actual sender was.

Example: Imagine a group of 100 people using a system like Tor to send messages. If an attacker sees a message and knows only that *someone* in that group sent it, then the **anonymity set size** is 100.

Key Properties:

- **Size matters**: Larger anonymity sets usually imply stronger anonymity.
- **Quality also matters**: If some users can be ruled out based on behavior, timing, or other side-channel data, the *effective* anonymity set becomes smaller.
- **Used in**: Mixnets, Tor, anonymous credentials, electronic voting, ring signatures, etc.

Website fingerprinting

Adversaries have become smarter, have been using ML to train classifiers on network traffic:

Mixer networks forced adversaries to come up with better tools.

Adversaries then moved to WS-fingerprinting, train ML classifiers on various network-traffic features such as pkt arrival, departure times, pkt sizes, inter-burst times, etc.

Even if the traffic is encrypted, the network information is enough for adversaries to link clients to destinations.

Website fingerprinting

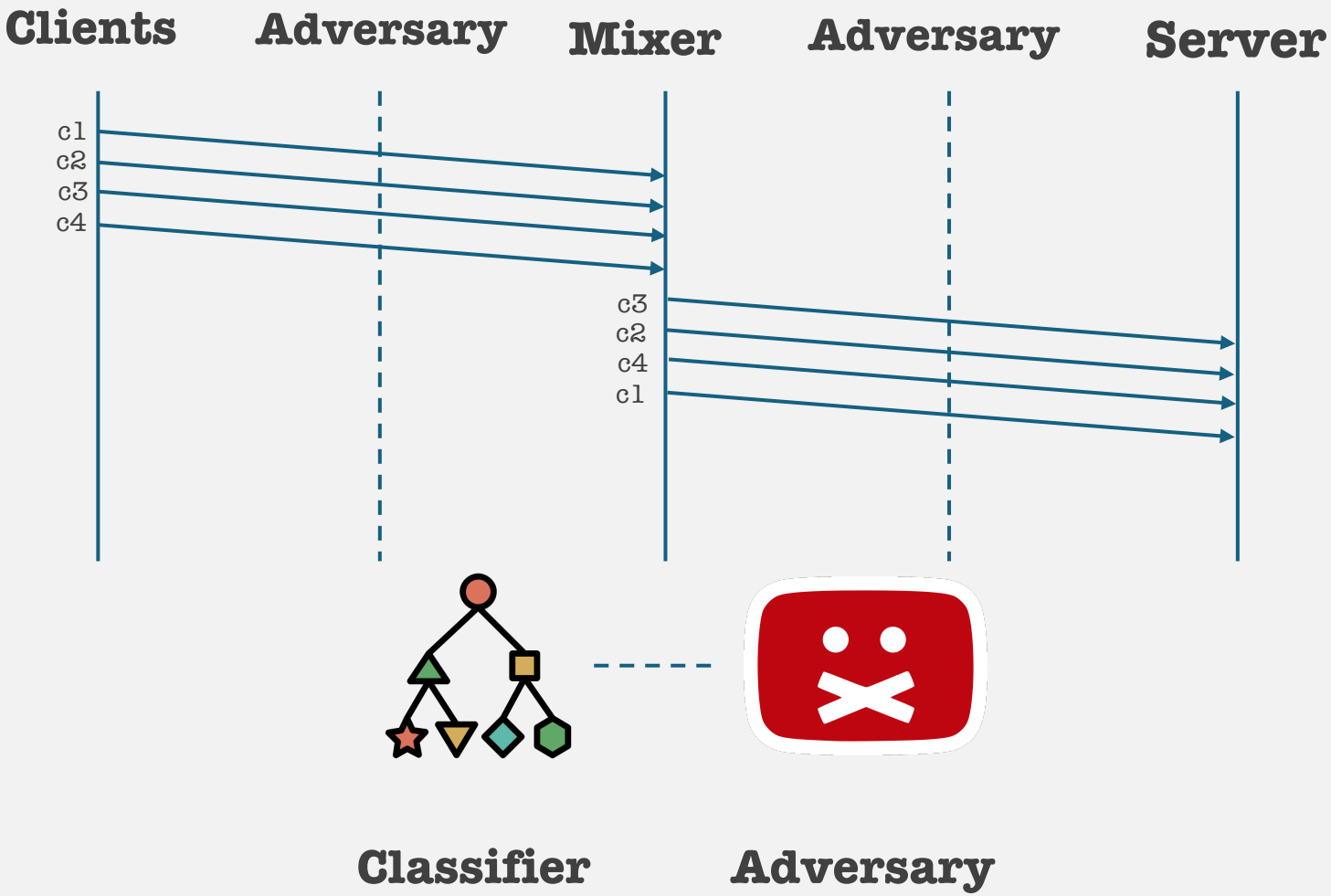
Adversaries have become smarter, have been using ML to train classifiers on network traffic:

Train models on known websites you wish to fingerprint.

(Facebook, Instagram, Telegram, Signal, etc)

Use trained models on real traffic to deanonymize src/dst.

Website fingerprinting



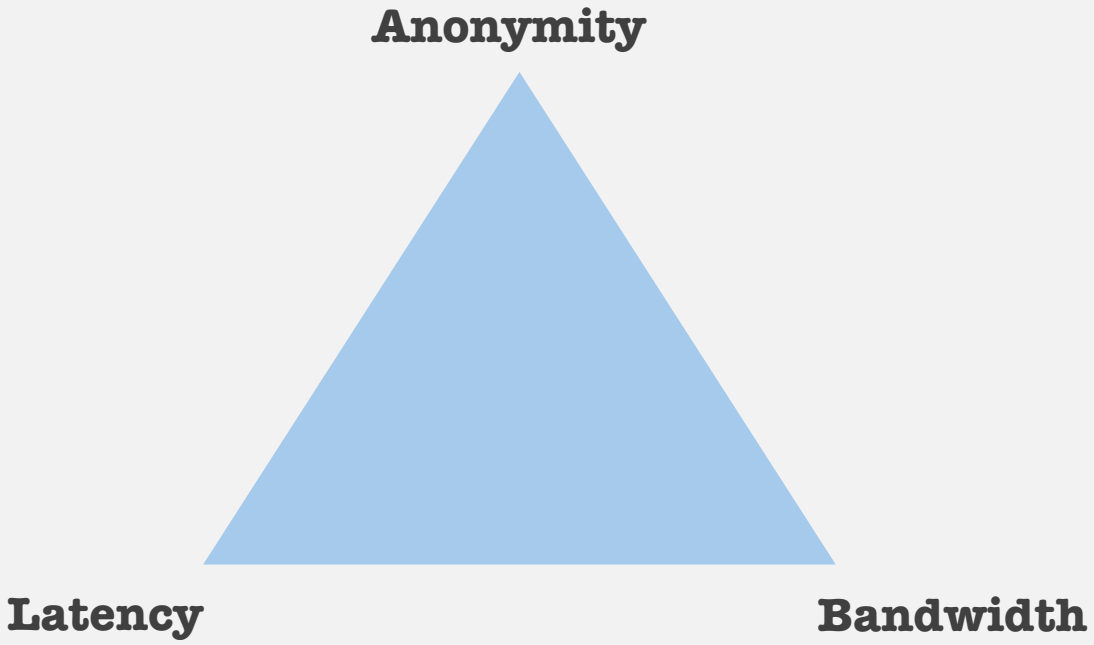
Classifier now ‘classifies’ the traffic and links clients to censored destination.

Key takeaways

1. The goal of privacy is to control the amount of information disseminated from your online communication.
2. The goal of anonymity is to prevent anyone from linking you to your desired content of interest.
    a. TLS encrypts traffic, therefore the only observable field on the internet that links src to dst is the IP address.
3. Anonymous communication assumes that the trusted entity may be your adversary.
4. Anonymous communication hinges on the assumption that **any** signal that links src to dst **must** be removed.
5. However.. There is a cost..

Anonymity trichotomy

Anonymous communication suffers from a fundamental trade-off..



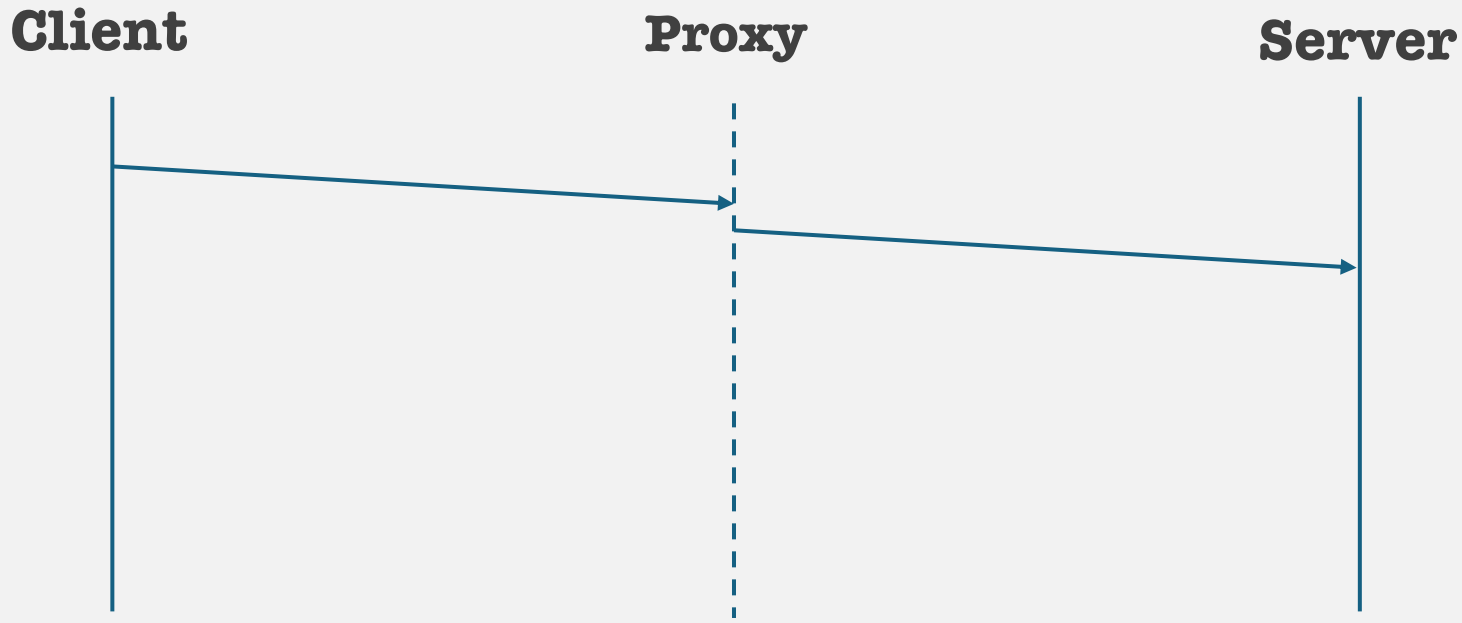
You can only pick 2..

Mixer networks give up **latency** to keep privacy.

Tor gives up on **bandwidth**.

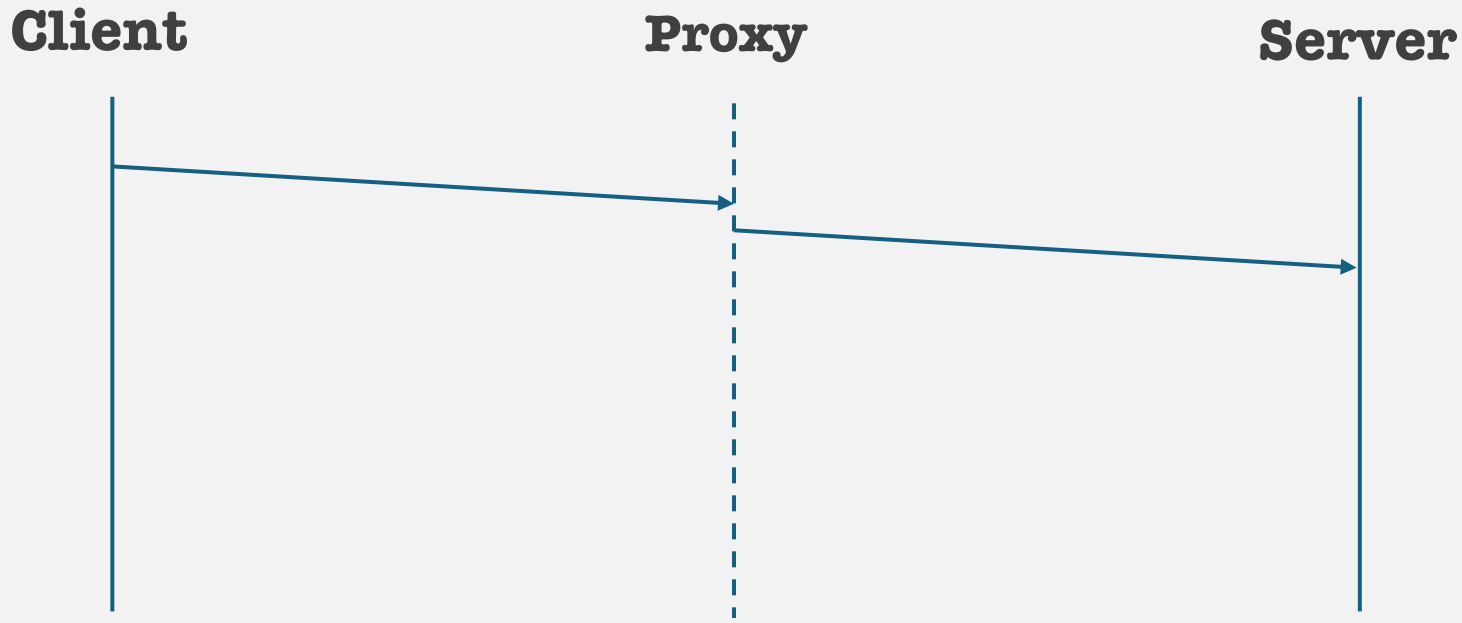
You must compromise on something..

The Onion Router: Tor



If the adversary controls the VPN/Proxy, ggwp

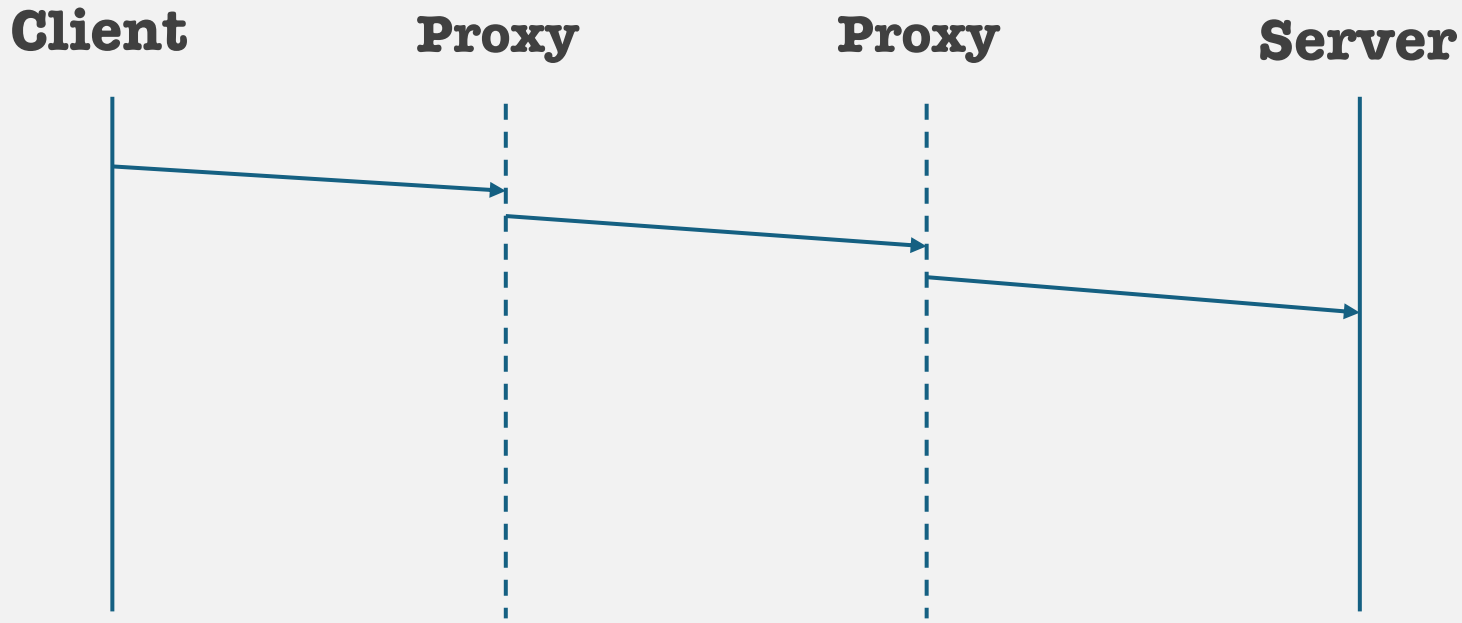
The Onion Router: Tor



If the adversary controls the VPN/Proxy, ggwp

What if you use proxy tunnels?

The Onion Router: Tor



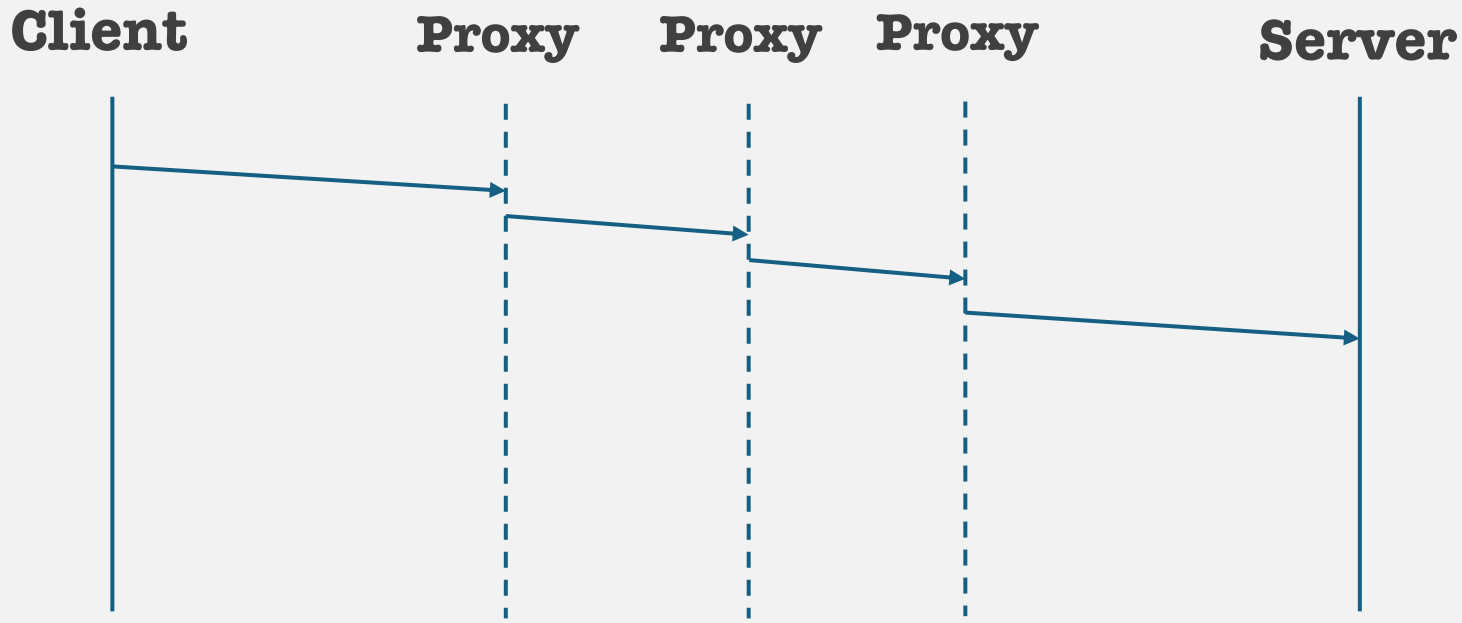
If the adversary controls the VPN/Proxy, ggwp

What if you use proxy tunnels?

If one of the proxies are adversary controlled, you're fine.

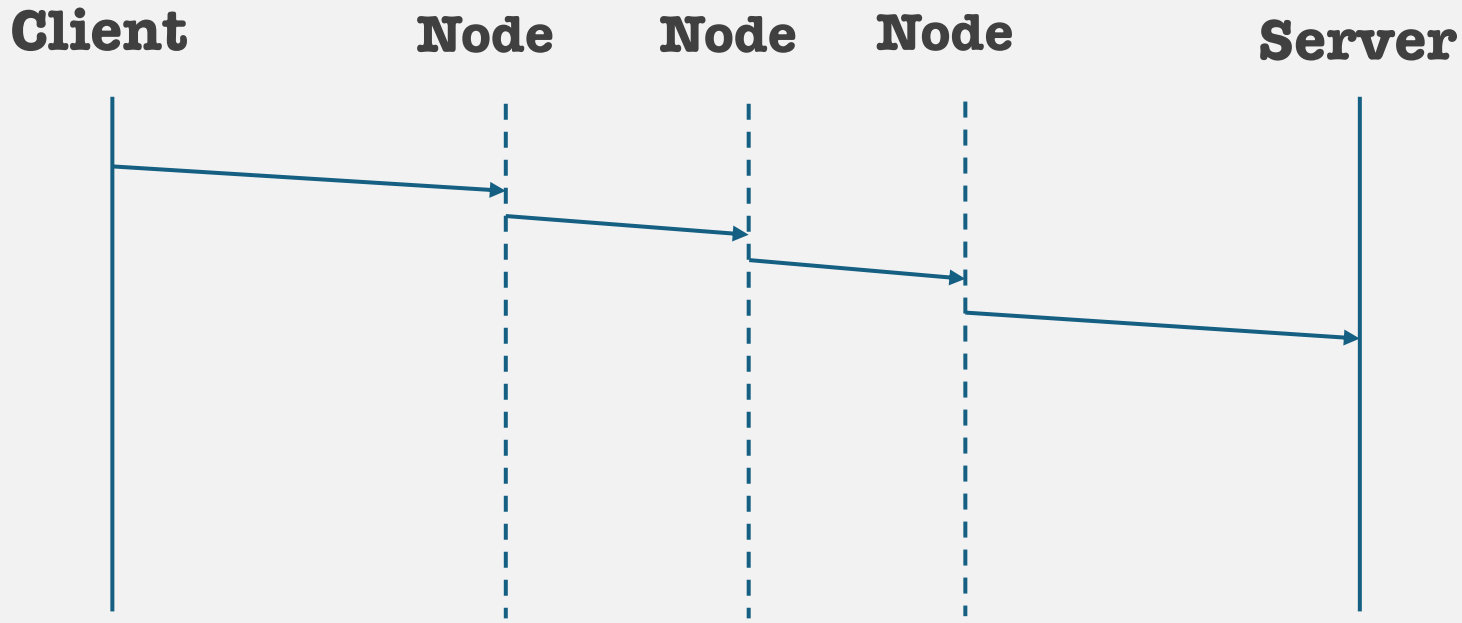
What if the two proxies collude?

The Onion Router: Tor



With three proxies, it's **very difficult to control all three.**

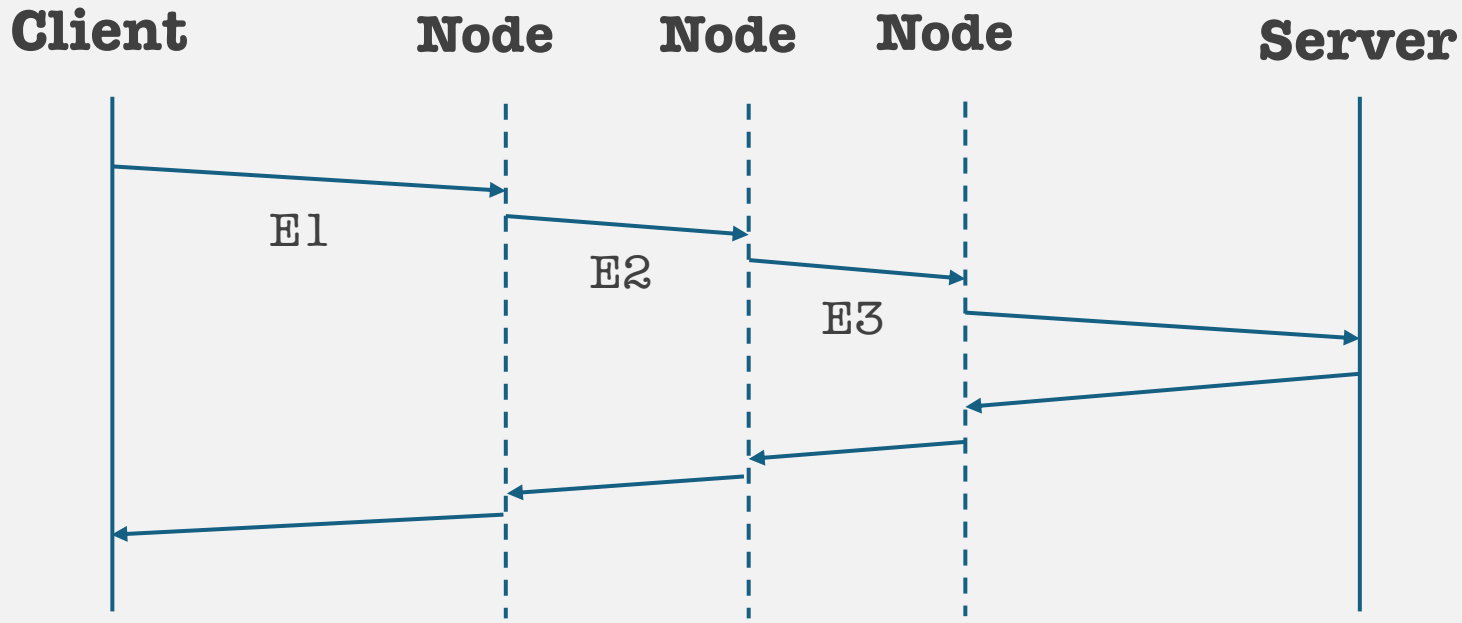
The Onion Router: Tor



With three proxies, it's **very difficult to control all three.**

Lets call the proxies **nodes**.

The Onion Router: Tor

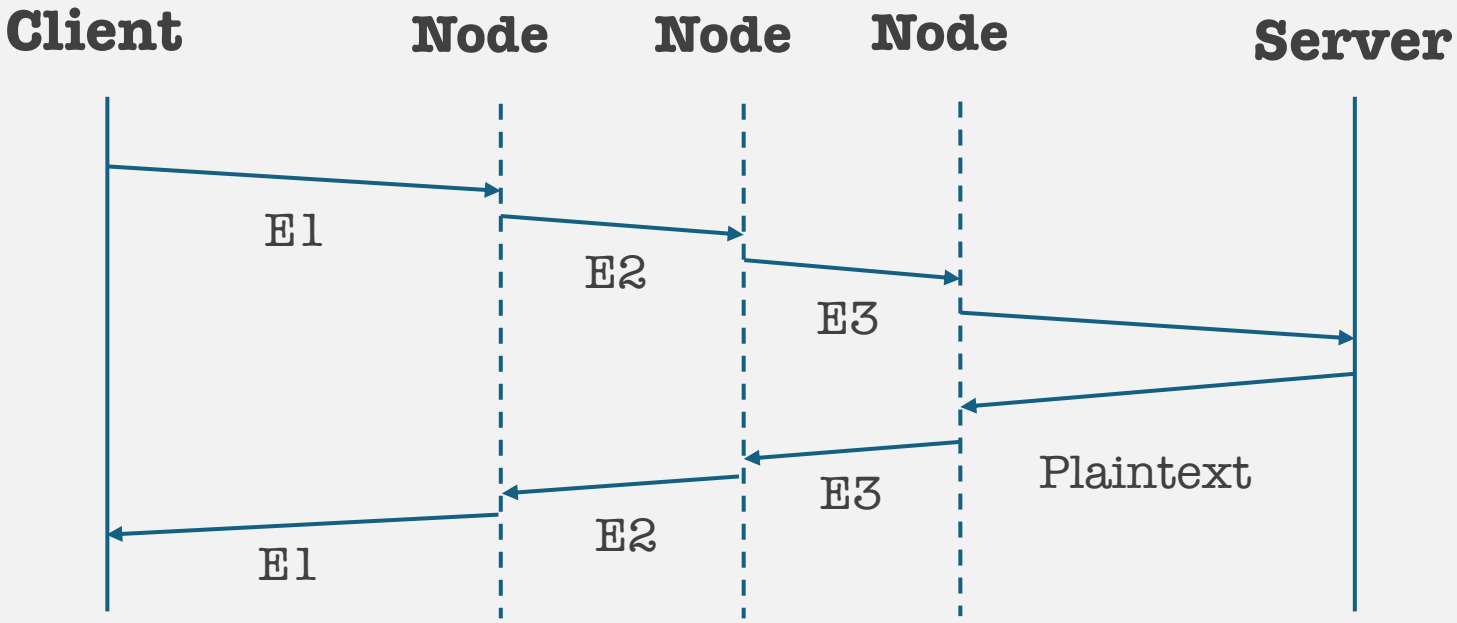


With three proxies, it's **very difficult to control all three.**

Lets call the proxies **nodes**.

Lets make the client encrypt things for each node.

The Onion Router: Tor



With three proxies, it's **very difficult to control all three.**

Lets call the proxies **nodes**.

Lets make the client encrypt things for each node. The nodes decrypt the packets with their keys and forward to the next node

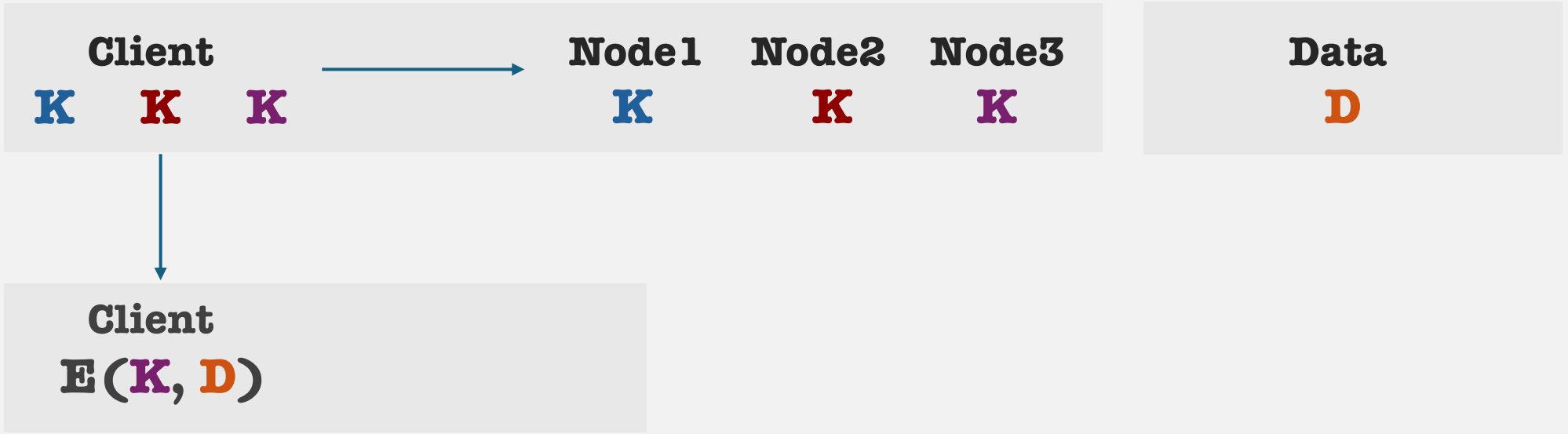
On the way back, each node encrypts the packets with their keys. The client can decrypt the entire encrypted message as the client has all three keys.

Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**

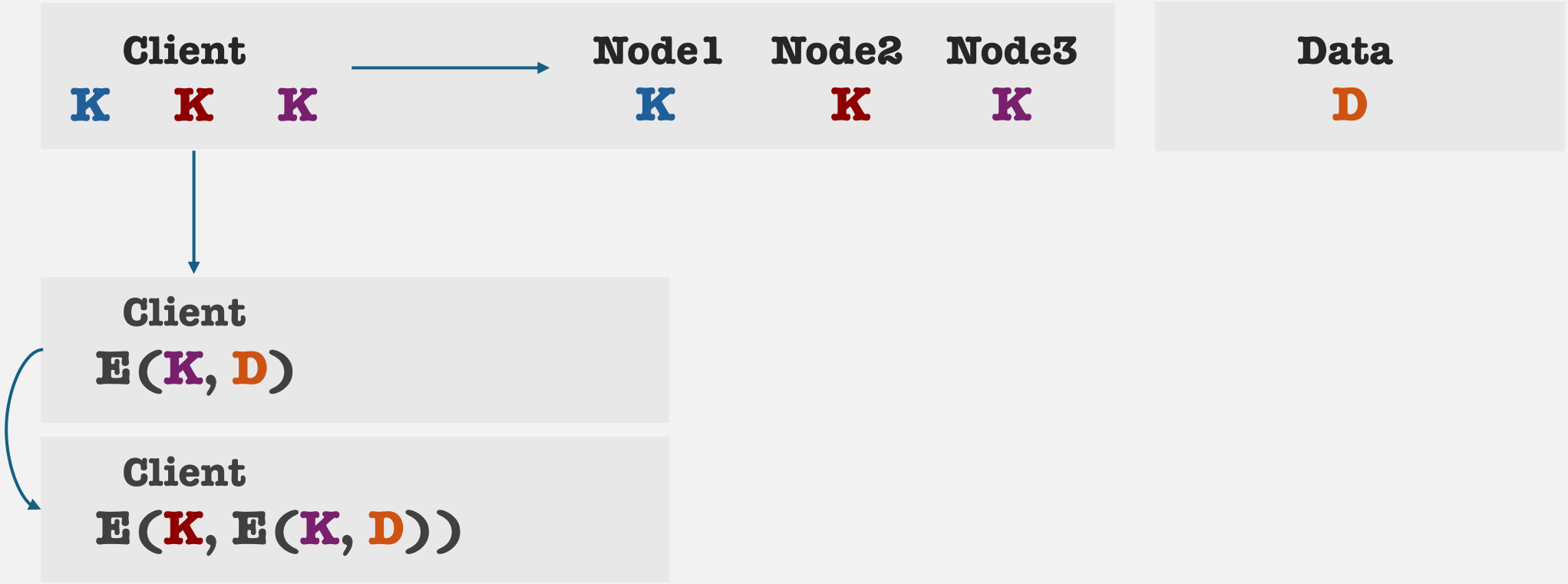
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



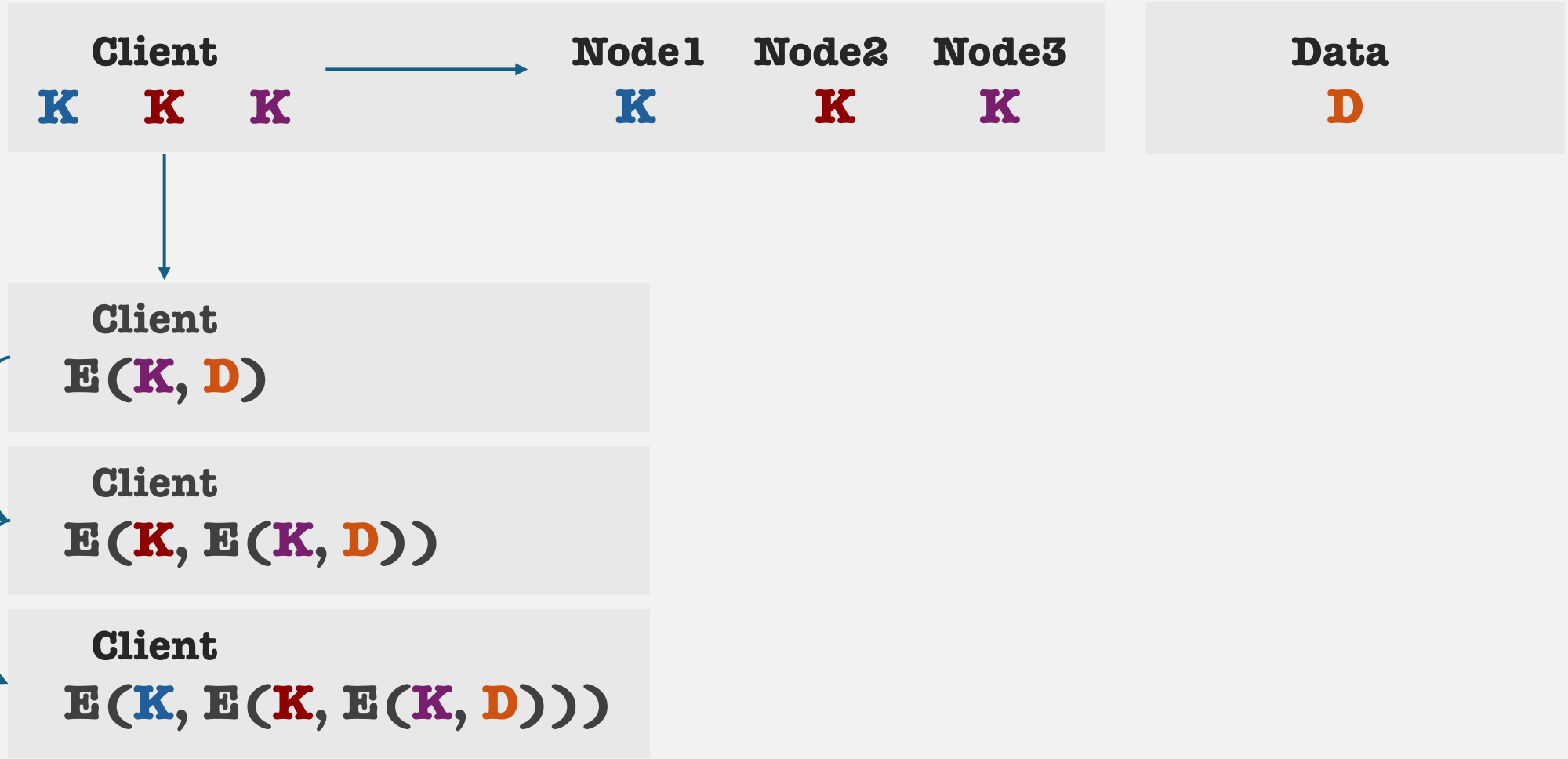
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



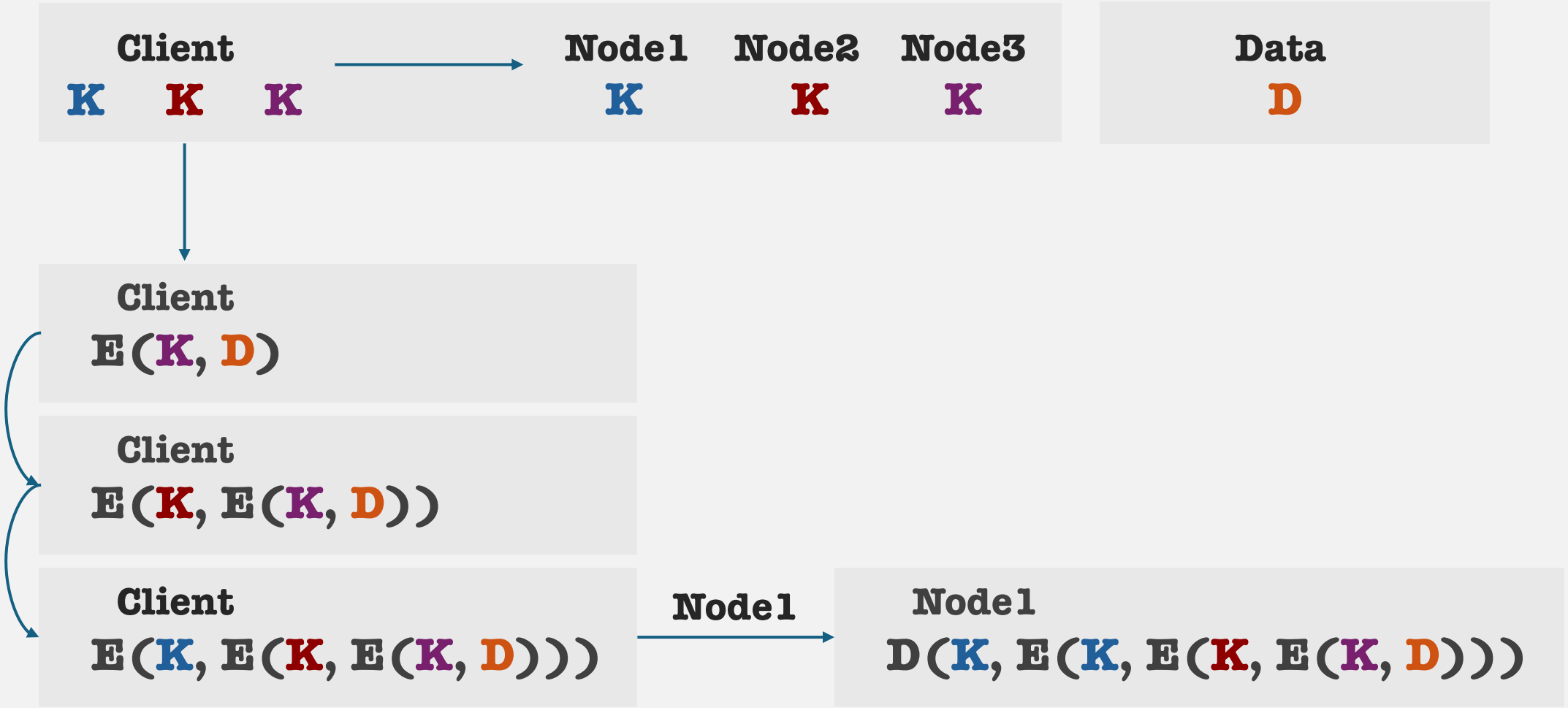
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



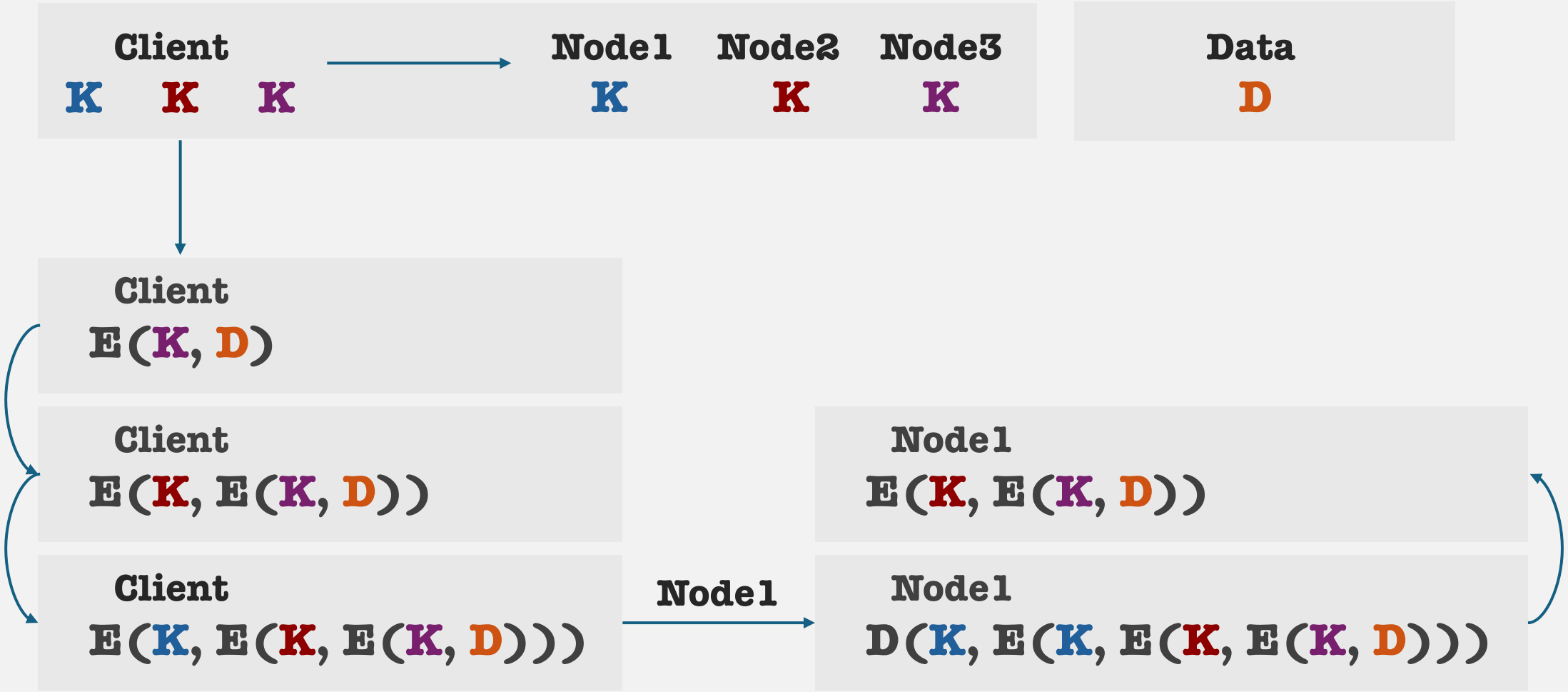
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



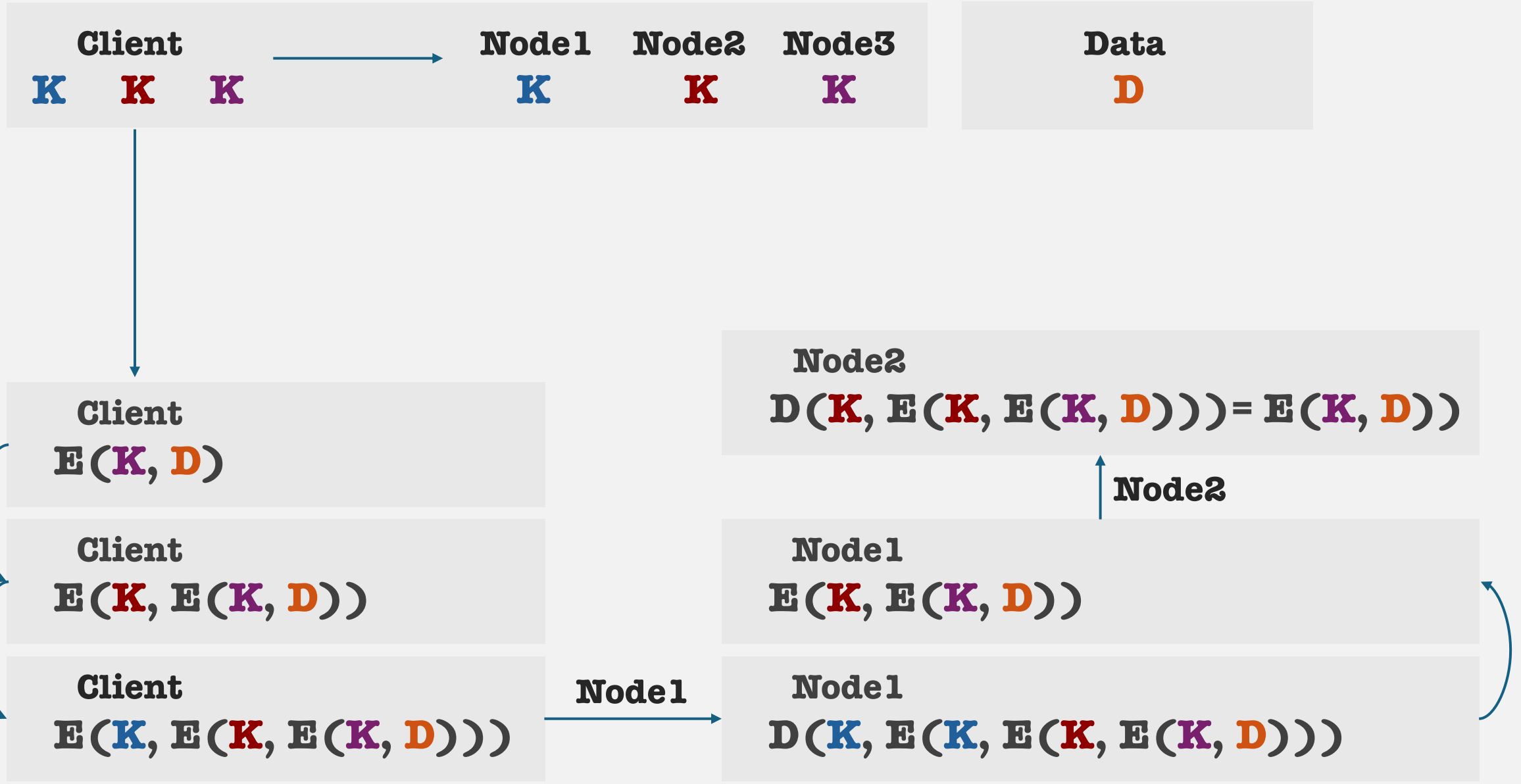
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



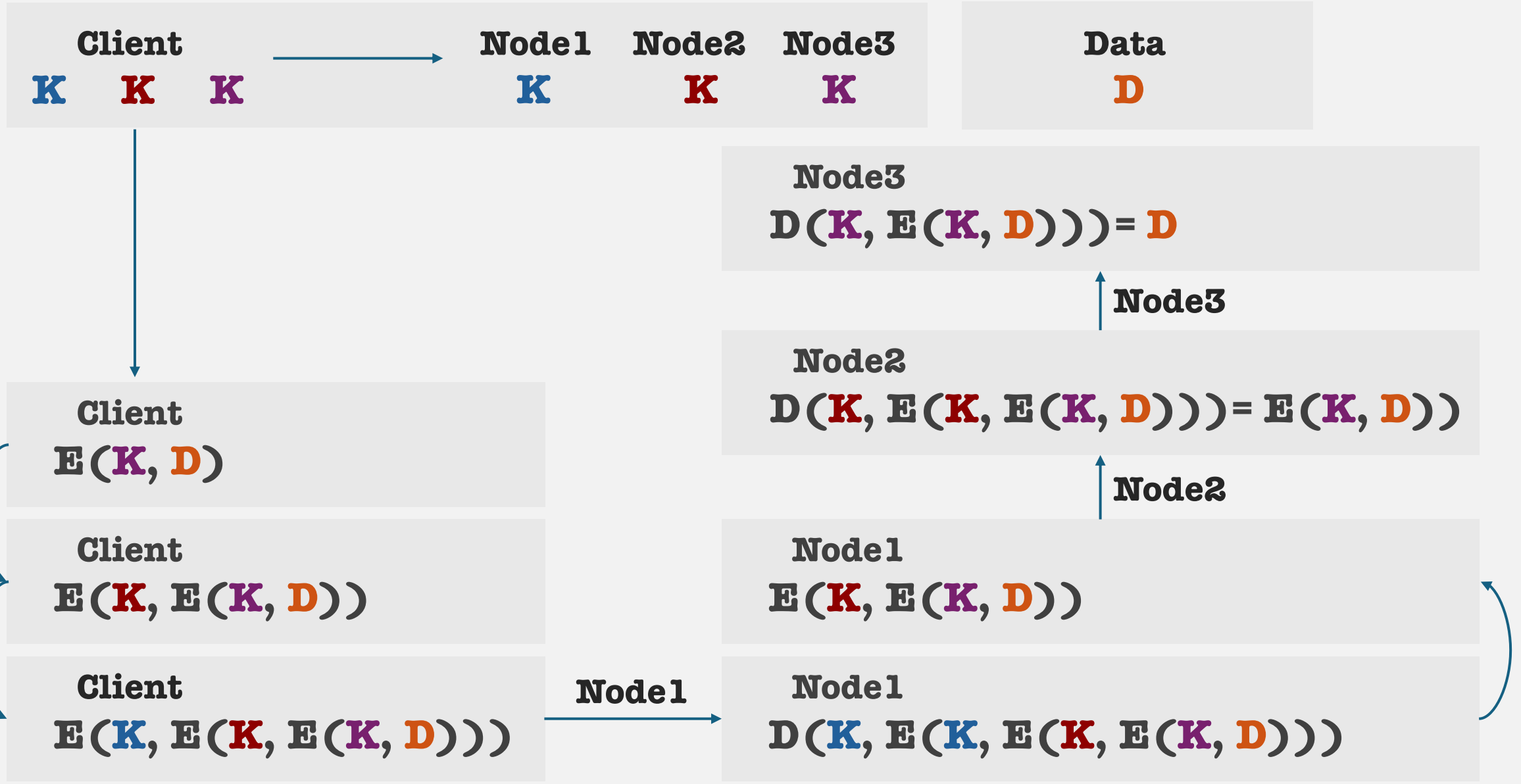
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



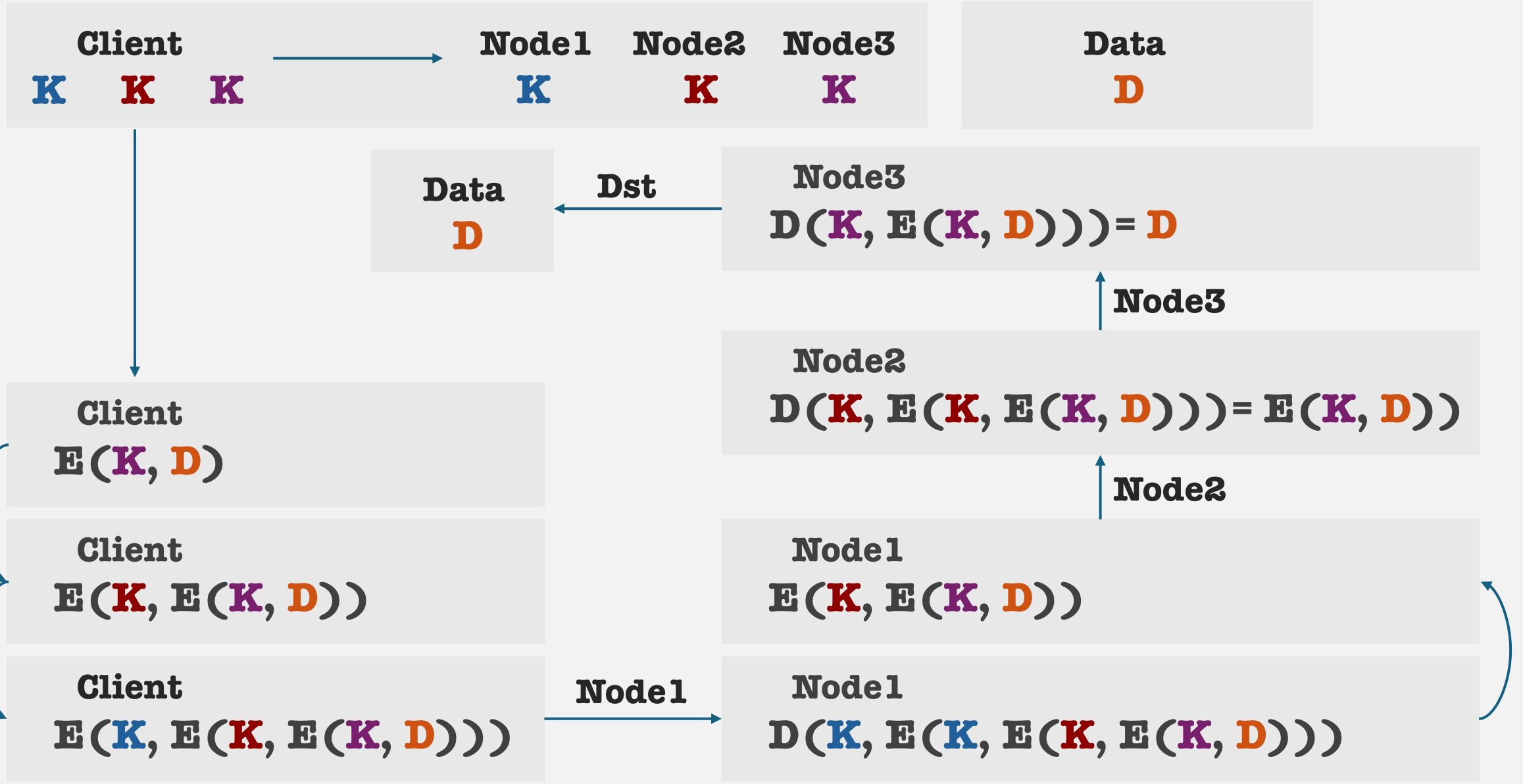
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



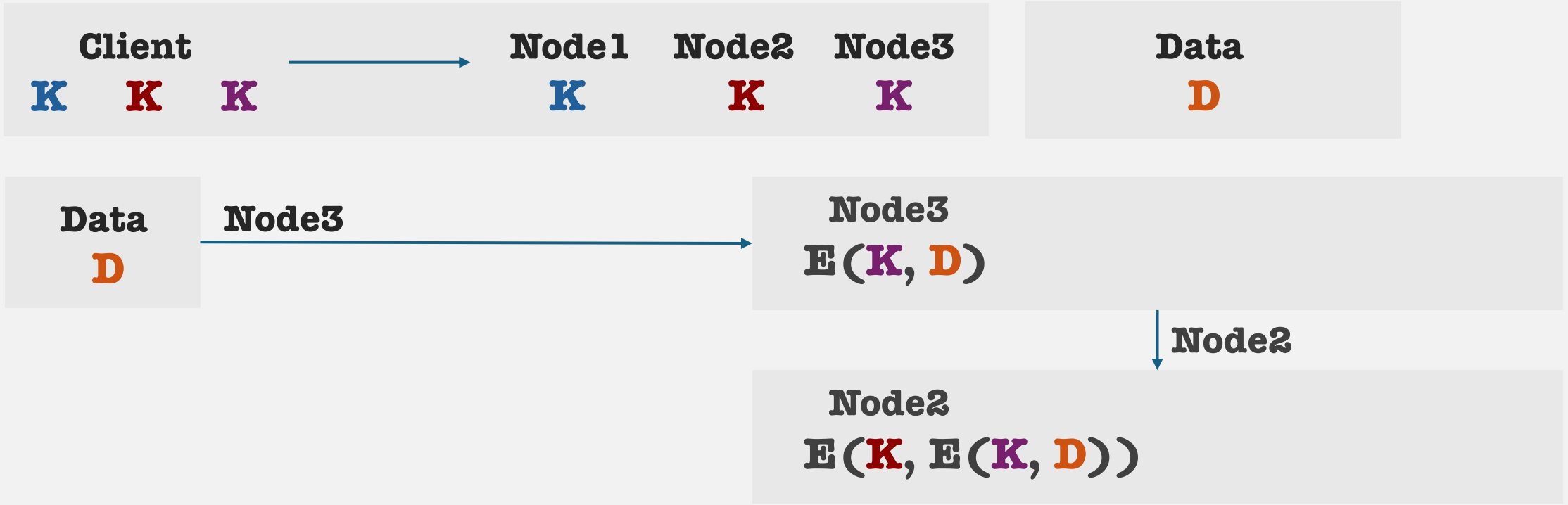
Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**

Reverse direction.

Onion routing

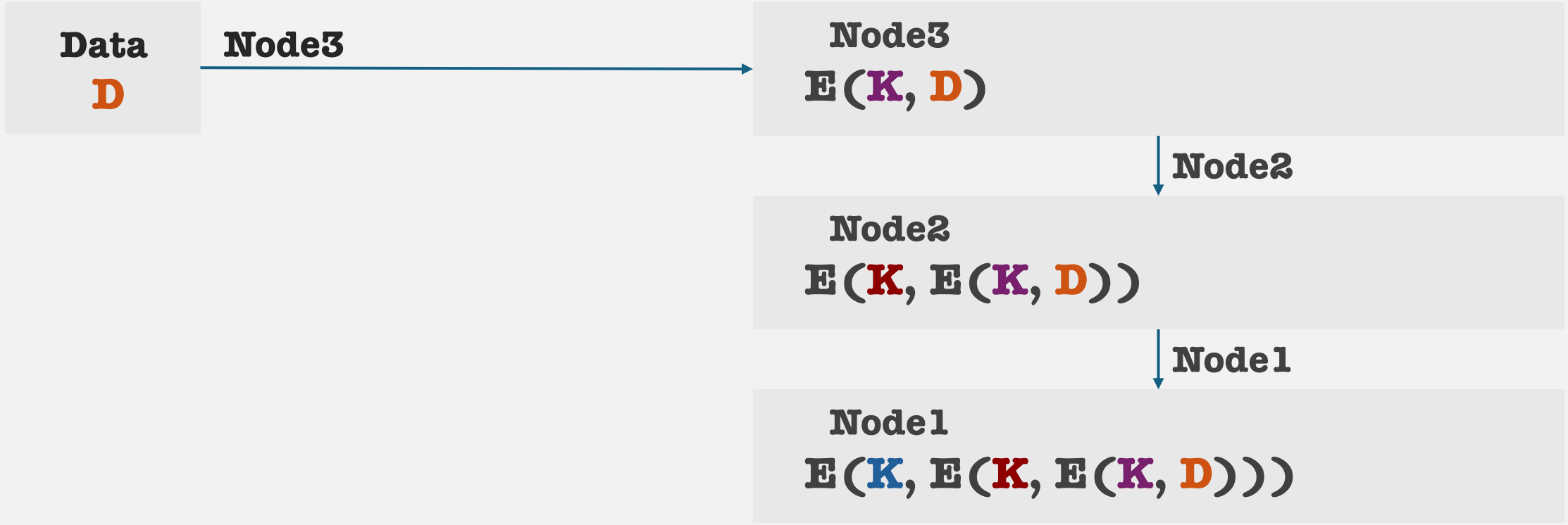
Think of this as **making an onion,** and then at each step **unpeeling the onion.**



Reverse direction.

Onion routing

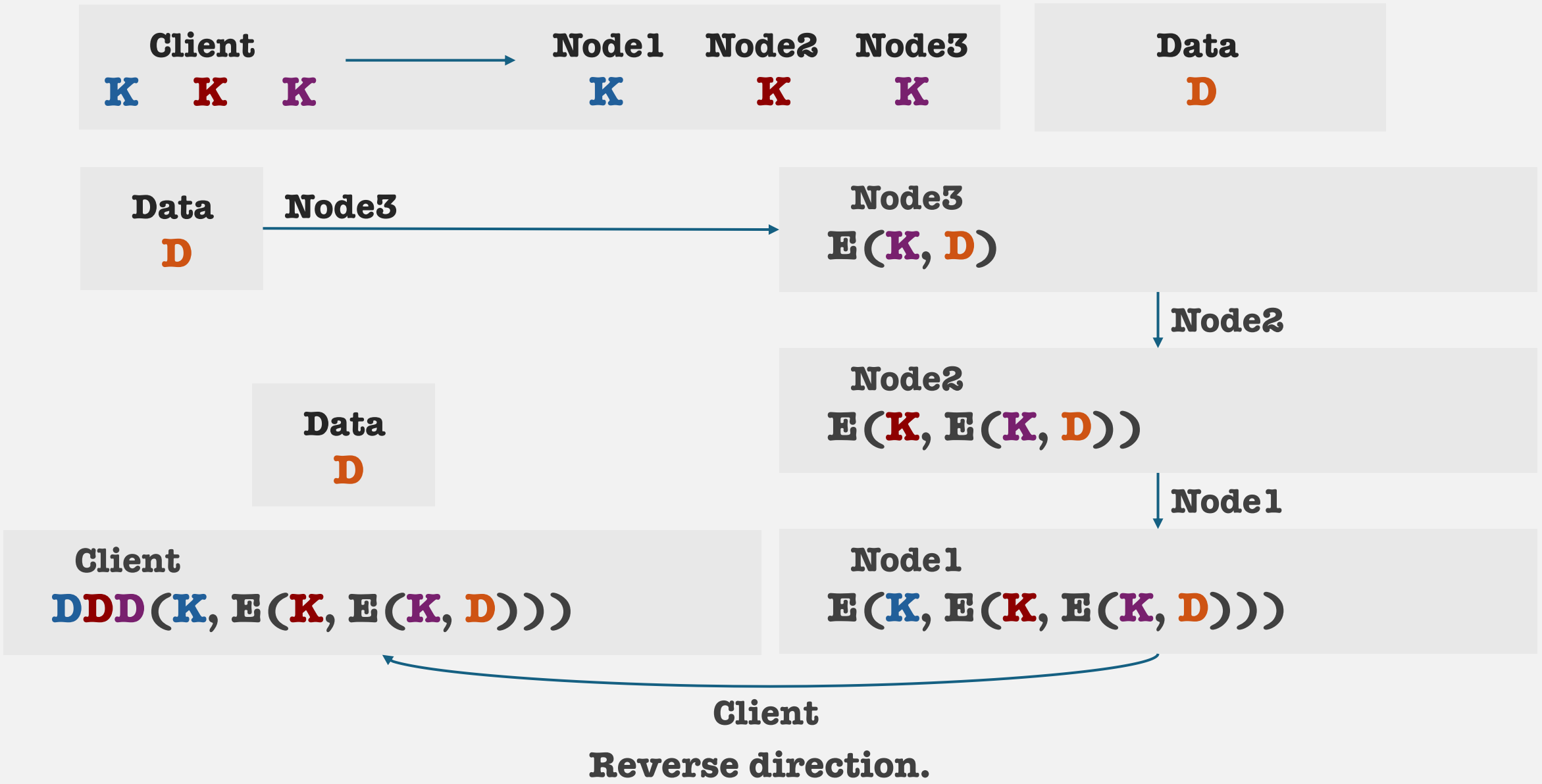
Think of this as **making an onion,** and then at each step **unpeeling the onion.**



Reverse direction.

Onion routing

Think of this as **making an onion,** and then at each step **unpeeling the onion.**



The Onion Router: Tor

1. Essentially, a **circuit** consists of **three** nodes, called **relays**, each of these nodes have a name:
    1. Entry guard (first relay) → This node **guards** the client from adversaries
    2. Middle relay (second relay)
    3. Exit node (third relay) -> This is where the client sent traffic **exits** the TOR network
2. Principle of encapsulation (and yes, including the IP headers)
3. Visible (source, destination) pairs visible on the network:
    1. First hop: (client, entry guard)
    2. Second hop: (entry guard, middle relay)
    3. Third hop: (middle relay, exit node)
    4. Fourth hop: (exit node, destination)
4. Reverse direction, everything flips

Circuit establishment

1. **Goal:** The client needs to establish a **route** to the destination, generate **ephemeral keys** with each relay.
2. **Approach:** The client fetches the list of public relays from Tor's public directory (trusted entities operated by Tor and select organizations)
    1. **Pick an entry guard**
        1. Establish a TLS connection
        2. Use the **entry guard's** long-term public-key to generate an **ephemeral key-pair**(asymmetric).
        3. This key-pair is used for generating a shared key with the entry (symmetric).
        4. Send/recv parts of the key to confirm the handshake.
    2. **Extend to middle relay**
        1. Same procedure, generate a key with the middle relay **(note: the entry guard simply forwards the packets and cannot obtain the key the client shared with the middle)**
    3. **Extend to exit node**
3. **Each node only sees their own shared key with the client.**

Circuit establishment

1. **Goal:** The client needs to establish a **route** to the destination, generate **ephemeral keys** with each relay.
2. **Approach:** The client fetches the list of public relays from Tor's public directory (trusted entities operated by Tor and select organizations)
    1. **Pick an entry guard**
        1. Establish a TLS connection
        2. Use the **entry guard's** long-term public-key to generate an **ephemeral key-pair**(asymmetric).
        3. This key-pair is used for generating a shared key with the entry (symmetric).
        4. Send/recv parts of the key to confirm the handshake.
    2. **Extend to middle relay**
        1. Same procedure, generate a key with the middle relay **(note: the entry guard simply forwards the packets and cannot obtain the key the client shared with the middle)**
    3. **Extend to exit node**
3. **Each node only sees their own shared key with the client.**
4. For looking up regular websites (https://google.com, https://facebook.com), the client can proceed to send and receive packets.

Q. What if websites want to hide their identity?

Hidden services

1. Adversaries wanting to block content can simply ban websites.
2. In the anonymity space, this is known as **censorship.**
3. The Tor Network extends anonymity to websites, these *hidden* websites are called **hidden services.**
4. Challenges:
    1. Can't use DNS
    2. Can't reveal IP addresses
    3. How are these services hosted?
    4. How are clients supposed to find these services?
    5. URLs are obfuscated e.g http://ryvveqz5chqxq5zxxc7zvqlcv5vfh7czmgeu5b5frvgf26a2g7zq2iad.onion

Good luck finding that URL

Hidden services: Setup

1. Hidden/Onion services established 3+ Tor circuits to random relays
2. These relays act as **introduction points.**
3. The IPs get the public-key and instructions from the hidden service.
4. These circuits **remain open.**
5. The service now creates a **descriptor** which has the following:
    1. **The IPs**
    2. **Authentication information**
    3. **Signatures**
    4. This descriptor is encrypted and signed by the hidden service.
6. This information is stored in a decentralized rotating-hash-ring (like a DHT).

Hidden services: Connection

1. Client needs to know the **.onion** URL
2. Computes the descriptor ID, searches the **hidden service directory (DHT).**
3. This gives client the **Introduction Points (IPs),** and other information need for key generation.
4. The client picks one of the **IPs** this IP is known as the **rendezvous point.**
5. The onion service and client now communication through their two circuits, through the rendezvous point.